



清华大学

综合论文训练

自动化 AI 口试系统的设计与实现

系 别： 计算机科学与技术系

专 业： 计算机科学与技术

姓 名： 关 世 开

指导教师： 向 勇 老师

二〇二六年五月

摘 要

近年来,大语言模型的普及对高等教育中的编程实验评估带来了严峻挑战,学生可轻易借助 LLM 生成高质量代码与实验报告,使得传统基于文本产出的评估方式逐渐失效。针对操作系统课程实验考核的特殊性与规模化瓶颈,本研究设计并实现了一套面向 OS 实验的自动化 AI 口试系统。该系统以“代码提交—智能出题—语音追问—多模型可信评分”为闭环,集成了基于 RAG 知识增强的自动出题模块、基于 WebSocket 的低延迟实时语音交互模块,以及“独立评分—交叉评审—主席裁定”三阶段多 LLM 协作评分模块,并引入基于统计离群值检测的定向重评机制以提升评分鲁棒性。实验采用受控模拟方法,在 ucore/rcore 六个实验章节上生成 48 组四级能力水平的模拟考生答案进行评测。结果表明,系统在 Spearman 秩相关层面达到 $\rho = 0.895$ 的强相关水平,相邻准确率达 97.92%,高/中置信度合计占比 72.92%,验证了其在粗粒度能力排序与区间划分上的可靠性。本研究为 LLM 时代计算机专业实验课程的自动化口试提供了一套可落地、可扩展的技术框架与实践参考。

关键词: AI 口试系统; 操作系统实验; 语音口试; 检索增强生成; 多模型评分

Abstract

The proliferation of large language models (LLMs) has posed severe challenges to programming experiment assessment in higher education, as students can easily leverage LLMs to generate high-quality code and lab reports, rendering traditional text-based evaluation increasingly ineffective. To address the unique characteristics and scalability bottlenecks of operating system (OS) course experiment assessment, this study designs and implements an automated AI-driven oral examination system tailored for OS experiments. The system establishes a closed-loop workflow of "code submission, intelligent question generation, voice interrogation, trustworthy multi-model scoring," integrating three core modules: a Retrieval-Augmented Generation (RAG) enhanced automatic question proposer, a low-latency real-time voice interaction module based on WebSocket, and a three-stage multi-LLM collaborative scoring framework comprising independent scoring, peer review, and chairman adjudication, supplemented by a statistical outlier detection and targeted re-evaluation mechanism to enhance scoring robustness. Using a controlled simulation approach, the system was evaluated on 48 simulated examinee samples across six OS experiment chapters at four proficiency levels. Experimental results demonstrate a strong Spearman rank correlation of $\rho = 0.895$, an adjacent accuracy of 97.92%, and a combined high/medium confidence ratio of 72.92%, confirming the system's reliability in coarse-grained ability ranking and interval classification. This research provides a deployable and scalable technical framework for automated oral examination of computer science laboratory courses in the LLM era.

Keywords: AI examination system; operating system lab; oral examination; retrieval-augmented generation; multi-model grading

目 录

第 1 章 研究背景与意义.....	1
1.1 研究背景与意义	1
1.1.1 大语言模型普及对传统编程作业评估的冲击	1
1.1.2 操作系统课程实验考核的特殊性与困难	1
1.1.3 AI 口试作为防作弊评估手段的必要性与可行性.....	2
1.1.4 构建自动化 AI 口试系统的教育价值与技术意义.....	2
1.2 国内外研究现状	3
1.2.1 基于 LLM 的自动出题与代码评估技术	3
1.2.2 多模型协作评估研究进展.....	3
1.2.3 语音交互在在线考试系统中的集成现状.....	3
1.2.4 RAG 技术在垂直领域教育问答中的应用	4
1.2.5 现有研究的不足与本研究的定位.....	4
1.3 研究内容与目标	4
1.3.1 核心研究问题界定.....	4
1.3.2 系统设计目标.....	5
第 2 章 相关技术.....	7
2.1 大语言模型技术	7
2.1.1 生成式预训练模型原理.....	7
2.1.2 基于提示词工程的代码分析.....	7
2.1.3 多 Agent 协作机制	8
2.2 检索增强生成 (RAG) 技术	8
2.2.1 RAG 基本架构	8
2.2.2 向量数据库与语义检索	8
2.3 语音处理技术	9
2.3.1 自动语音识别 (ASR) 原理与开源方案.....	9
2.3.2 文本转语音 (TTS) 技术.....	9
2.3.3 实时语音流处理与 WebSocket 通信.....	9
2.4 多模型集成评估方法	10
2.4.1 单模型评估的局限性.....	10
2.4.2 多模型投票与一致性检验方法.....	10

2.4.3 离群值检测与鲁棒性评估	10
2.5 现有技术总结	11
第 3 章 系统设计	12
3.1 需求分析	12
3.1.1 功能需求	12
3.1.2 非功能需求	13
3.1.3 用户角色分析	13
3.2 系统架构设计	14
3.2.1 前后端分离架构	14
3.2.2 核心模块划分	14
3.2.3 技术栈选型与部署架构	15
3.3 核心业务流程设计	16
3.4 数据模型与接口设计	16
3.4.1 核心实体定义	16
3.4.2 前后端关键 API 设计	17
第 4 章 RAG 知识库与自动出题模块	18
4.1 操作系统课程知识库构建	18
4.1.1 知识来源与数据预处理	18
4.1.2 文档切分与语义分块策略	18
4.1.3 向量嵌入模型选择与 ChromaDB 存储方案	19
4.1.4 课程适配机制	19
4.2 基于 RAG 的知识增强出题	20
4.2.1 检索策略设计	20
4.2.2 提示模板构建	20
4.2.3 代码片段关联机制	21
4.3 出题质量控制	21
4.3.1 问题互异性检查机制	21
4.3.2 问题质量过滤	21
第 5 章 多 LLM 协作评分模块	23
5.1 评分框架总体设计	23
5.1.1 多阶段协作思想	23
5.1.2 评分维度定义	23

5.2 第一阶段：独立评分与报告生成	24
5.2.1 多 LLM 并行评分架构	24
5.2.2 评分提示工程与约束设计	24
5.2.3 结构化评分报告生成	24
5.3 第二阶段：交叉评审与一致性检验	24
5.3.1 模型间交叉评审机制	24
5.3.2 评分一致性度量方法	25
5.3.3 置信度初步计算	25
5.4 第三阶段：综合裁决与结果生成	26
5.4.1 元评审模型设计	26
5.4.2 融合策略	26
5.4.3 最终评分报告与评语生成	26
5.5 离群值检测与重评机制	27
5.5.1 统计离群值识别方法	27
5.5.2 动态重评触发条件与流程	27
5.5.3 重评结果融合策略	28
5.6 评分可解释性设计	28
5.6.1 评分依据追溯	28
5.6.2 多维度展示	28
第 6 章 语音口试模块设计与实现	29
6.1 语音交互架构	29
6.1.1 实时语音流处理架构	29
6.1.2 语音状态机设计	29
6.2 语音识别与合成集成	30
6.2.1 ASR 模块集成	30
6.2.2 TTS 模块集成与音色选择	31
6.2.3 音频数据预处理与降噪	31
6.3 口试考试流程实现	31
6.3.1 考试界面与状态同步	31
6.3.2 题目语音播报与代码展示	31
6.3.3 考生语音回答采集与转写	32
6.3.4 超时处理与异常恢复机制	32

6.4 端到端响应延迟特征分析	33
6.4.1 延迟分解与测量方法	33
6.4.2 延迟测量结果	33
6.4.3 延迟分析	33
第 7 章 系统测试与实验评估	34
7.1 测试环境与数据集构建	34
7.1.1 测试环境	34
7.1.2 测试数据集构建	34
7.2 RAG 功能测试	34
7.2.1 RAG 延迟测试	34
7.2.2 RAG 检索覆盖率测试	35
7.3 评分系统评价体系	36
7.3.1 评价指标设计	36
7.3.2 模拟答案生成方法	37
7.4 实验结果与数据分析	38
7.4.1 实验结果	38
7.4.2 描述性统计与得分分布分析	38
7.4.3 评分区分度分析	39
7.4.4 评分一致性分析	40
7.4.5 系统鲁棒性分析	41
7.4.6 综合性评价	41
第 8 章 总结与展望	43
8.1 研究工作总结	43
8.2 不足与展望	43
8.2.1 RAG 知识库覆盖范围限制	43
8.2.2 评分标准的主观性残留	44
8.2.3 未来工作方向	44
参考文献	45
附录 A 模型列表	47
附录 B 系统提示词模板汇编	51
致 谢	60
声 明	61
在学期间参加课题的研究成果	62

插图清单

图 3.1 系统核心模块处理流程	15
图 6.1 端到端响应延迟阶段分解	33
图 7.1 RAG 延迟测试	34
图 7.2 RAG 检索覆盖率总体指标	36
图 7.3 各真实等级样本的得分分布。(a) 箱线图展示四分位距与异常值；(b) 均值 $\pm$ 标准差柱状图。A 与 B 等级的均值差异极小 ($\Delta = 0.17$)，且标准差区间重叠。	38
图 7.4 (a) 混淆矩阵显示 B $\rightarrow$ A 与 C $\rightarrow$ B 的误判为主要误差来源；(b) 各章节单调性趋势，ch8 出现 A-B 反转	39
图 7.5 评分一致性与系统鲁棒性分析	41

附表清单

表 7.1 各真实等级样本的得分描述性统计	39
-----------------------------	----

第 1 章 研究背景与意义

1.1 研究背景与意义

1.1.1 大语言模型普及对传统编程作业评估的冲击

近年来，以 GPT、Claude、Gemini 为代表的大语言模型在代码生成、文本润色与知识问答等任务上展现出接近甚至超越人类专家的能力。这一技术突破在显著提升生产效率的同时，也对高等教育中的学术诚信与教学评估体系带来了前所未有的挑战。在编程类课程中，学生仅需向 LLM 提供自然语言描述，即可在数秒内获得结构完整、语法正确的代码实现；更有甚者，部分学生直接提交由 LLM 生成的实验报告，其行文流畅度与逻辑严密性往往令教师难以仅凭肉眼判别真伪。

传统的代码相似性检测工具（如 MOSS）主要防范学生之间的互相抄袭，却难以应对“人机协作”或“完全代写”的新型作弊模式。Gacek 等人针对算法与数据结构课程的研究表明，即便学生仅对 LLM 生成的代码进行变量重命名或格式微调，基于代码嵌入的语义相似性检测仍能在一定程度上识别出高度可疑的提交^[1]。然而，检测系统只能标记风险，无法衡量学生是否真正理解代码背后的设计思想。正如纽约大学 Stern 商学院的 Panos Ipeirotis 教授在其教学实践中的观察：许多提交了“thoughtful, well-structured work”的学生，在课堂随机追问下却无法解释其提交作品中的基本设计选择，甚至完全无法作答。这揭示了一个根本性问题——当书面作业可以被 LLM 高质量代劳时，传统的基于文本产出的评估方式正在逐渐失效^[2]。

1.1.2 操作系统课程实验考核的特殊性与困难

操作系统课程是清华大学计算机系本科阶段的核心必修课，其实验环节要求学生深入理解进程管理、内存管理、中断处理、文件系统等底层机制，并在 ucore 或 rcore 教学操作系统内核框架上完成代码补全与功能扩展^[3]。与其他编程课程相比，OS 实验具有三个显著特点：其一，代码上下文高度复杂，单个实验往往涉及数十个源文件、数百个函数调用关系，学生需要对内核整体架构有宏观把握；其二，实验答案具有强开放性，同一道题目可能存在多种实现路径，教师难以通过简单的测例或代码查重完成评判；其三，实验强调对底层机制的理解而非单纯的代码编写，例如页替换算法的设计决策、中断上下文的保存与恢复流程等，均需要学生具备扎实的概念认知与调试能力。

这些特点使得 OS 实验成为 LLM 辅助作弊的“重灾区”：学生可以轻易让

LLM 生成一段“看起来正确”的代码片段，甚至借助 LLM 撰写一份详尽的实验报告，但其对时钟中断处理流程、页表项与物理页映射关系等核心知识点的理解可能极为薄弱。与此同时，由于班级规模较大，教师与助教难以对每一份实验代码进行逐行审阅与面对面追问，导致评估质量与效率之间的矛盾尤为突出。

1.1.3 AI 口试作为防作弊评估手段的必要性与可行性

面对 LLM 对书面评估的侵蚀，教育界开始重新审视口试这一传统评估形式的价值。口试要求考生在实时交互中展示其对知识的理解、逻辑推理与即兴表达能力，这种“实时性”与“交互性”天然提高了 LLM 代答的门槛——考生无法像完成书面作业那样，在不受监督的环境下通过多轮对话让 LLM 代为组织答案。然而，传统口试长期以来面临一个根本性的可扩展性瓶颈：以 36 名学生、每场 25 分钟、两名考官计算，仅考试与评分环节就需要消耗约 30 小时的教师工时；当班级规模扩大至百人以上时，人工口试几乎不现实。

语音 AI 技术的成熟为破解这一困境提供了新的可能。Ipeirotis 与 Rizako 在 2025 年底至 2026 年初开展了一项具有里程碑意义的教学实验：他们利用 Eleven-Labs 对话式 AI 平台构建语音考官代理，为 NYU 的“AI/ML 产品管理”课程实施了 36 场自动化口试，总成本仅为 15 美元（约合每名学生 0.42 美元）并实现了三模型协作评分与结构化反馈生成。这一思路不仅在经济成本上具有颠覆性，更在教育理念上具有深远意义：由于 LLM 可根据评分细则动态生成问题，考试结构本身可以完全公开，学生通过反复练习系统来备考而“练习即学习”，从根本上消除了泄题风险^[2]。

1.1.4 构建自动化 AI 口试系统的教育价值与技术意义

将语音 AI 口试技术引入操作系统课程实验评估，具有双重价值。在教育层面，它有望建立一种“代码提交 + 即时口试”的闭环评估机制：学生上传实验代码后，AI 考官不仅针对代码实现细节进行追问，还能结合操作系统核心概念展开深度探查，从而有效区分“亲手实现并理解”与“复制粘贴 LLM 输出”两类学生。在技术层面，该场景对现有 AI 评估技术提出了新的挑战与要求：操作系统领域的专业知识需要构建专门的检索增强生成知识库以提升出题与追问的专业性；实验代码的复杂性与开放性要求评分系统具备多维度、可解释的评判能力；语音交互的实时性则对系统架构的并发处理与稳定性提出了工程层面的要求。

因此，本研究旨在设计并实现一套面向操作系统课程实验的 AI 口试系统，集成自动出题、实时语音交互、多 LLM 协作评分与置信度检验等核心功能，以期为 LLM 时代的编程课程评估提供一种可落地、可扩展、可信赖的技术方案。

1.2 国内外研究现状

1.2.1 基于 LLM 的自动出题与代码评估技术

自动出题 (Automatic Question Generation, AQG) 是自然语言处理与教育技术领域的经典研究方向。传统方法主要依赖模板填充或语法规则, 生成的问题在灵活性与上下文关联性上存在明显局限。LLM 的出现显著提升了 AQG 的质量: 通过精心设计的提示词工程, 模型能够基于给定的知识点、代码片段或学生作业生成具有针对性与层次感的问答题目。在编程教育场景中, LLM 不仅可以生成概念性问题, 还能针对学生提交的代码提出特定的调试追问, 例如要求解释某一行关键代码的设计意图或边界条件处理逻辑。

在代码评估方面, 现有研究主要沿两条路径展开。一是静态检测路径, 通过代码嵌入、抽象语法树 (AST) 比对或风格特征提取, 识别 LLM 生成或改写过的代码提交^[1]。二是动态执行路径, 利用测试用例验证代码功能正确性。然而, 前者在识别的可靠性和泛用性上仍有待验证。后者则侧重于“代码本身”而非“代码作者的理解”, 无法回答“学生是否真正掌握了这段代码”这一教育评估的核心问题。

1.2.2 多模型协作评估研究进展

单一 LLM 在评判开放式任务时往往存在评分偏差与标准漂移问题。为提升评估的鲁棒性与一致性, 研究者提出了多模型协作评估框架, 即让多个不同架构或不同训练数据的大模型独立评分, 再通过某种聚合机制得出最终结论。Andrej Karpathy 提出的“council of LLMs”思想是该方向的代表性方案之一^[4]。

Ipeirotis 与 Rizakos 在其语音口试系统中将这一思想具体化为三阶段评分流程: 第一阶段, Claude、Gemini 与 GPT 三个模型独立阅读考试转录文本并给出各维度评分; 第二阶段, 每个模型审阅其余两个模型的评分依据与引用证据, 并据此修订自身评分; 第三阶段, 由指定的“主席模型”综合所有信息生成最终成绩与结构化反馈。实验数据显示, 经过协商修订后, 评分者间一致性的 Krippendorff's α 系数达到 0.86, 显著高于传统阈值。不过, 该研究也指出, 当学生回答本身含糊不清时, 模型间仍会在“部分得分”的判定上存在分歧, 这提示多模型评估机制仍需配合置信度检验与人工审计^[2]。

1.2.3 语音交互在在线考试系统中的集成现状

语音作为最自然的人机交互方式之一, 近年来随着端到端语音识别 (ASR) 与神经网络语音合成 (TTS) 技术的成熟, 逐渐被引入在线教育场景。ElevenLabs、Google Cloud Speech-to-Text 等平台已将 ASR、TTS、话轮转换与打断处理等能

力打包为对话式 AI 服务，使得构建语音考官的技术门槛大幅降低。在招聘面试场景中，AI 主持的语音面试在质量上可以达到甚至超越人工面试的水平，这为语音评估技术在高风险教育场景中的应用提供了信心。

尽管如此，现有语音考试系统多聚焦于通用知识问答或商业案例分析，鲜有针对计算机专业编程实验的垂直场景设计。操作系统实验考核要求语音系统能够理解并播报技术术语、处理代码片段的朗读与展示，并在考生回答涉及底层实现细节时进行有效追问，这对语音交互的领域适配性提出了更高要求。

1.2.4 RAG 技术在垂直领域教育问答中的应用

检索增强生成 (RAG) 通过在外部知识库中检索与查询相关的文档片段，并将其作为上下文注入 LLM 的生成过程，有效缓解了模型幻觉问题，并提升了专业领域问答的准确性^[5]。在教育场景中，RAG 已被用于构建课程知识库、辅助答疑与生成学习材料^[6]。对于操作系统课程而言，RAG 知识库可以收录实验指导书、内核源码注释、历年优秀实验报告以及常见错误案例，从而为自动出题与深度追问提供领域知识支撑。

1.2.5 现有研究的不足与本研究的定位

综合上述分析，当前研究存在以下不足：第一，现有的 LLM 代码检测与自动评估工具侧重于“识别抄袭”，缺乏对“理解深度”的测评手段；第二，已有的语音 AI 口试系统主要面向商业与管理类课程，尚未针对操作系统等底层系统类课程的实验特点进行专门设计；第三，多模型协作评分框架虽已初步验证其有效性，但在评分离群值检测、置信度量化与自动重评机制方面仍缺乏系统性研究；第四，RAG 技术与语音口试系统的结合尚处于概念阶段，缺乏完整的工程实现与教学实验验证。

本研究正是在上述背景下展开，定位于填补“垂直领域 + 语音交互 + RAG 知识增强 + 多模型可信评分”这一交叉空白，设计并实现一套完整的 AI 口试系统，以期为 LLM 时代的计算机专业实验课程评估提供可复用的技术框架与实践参考。

1.3 研究内容与目标

1.3.1 核心研究问题界定

本研究围绕大语言模型时代操作系统课程实验评估所面临的现实困境展开，核心研究问题可归纳为以下四个层面：

- (1) LLM 普及背景下 OS 实验代码的真实性评估困境。随着 GPT、Claude 等

通用大语言模型在代码生成任务上的能力跃升，学生可轻易获取与实验要求高度匹配的代码实现，甚至借助 LLM 撰写完整的实验报告。现有基于代码相似性检测（如 MOSS）或静态特征嵌入的抄袭识别工具，仅能标记“人机协作”的风险痕迹，却无法判定学生是否真正理解代码背后的内核机制。这导致传统以书面代码提交为核心的评估方式逐渐失效，教师难以区分“亲手实现并深刻理解”与“复制粘贴 LLM 输出”两类学生。

(2) 传统口试在 OS 实验场景中的可扩展性瓶颈。口试通过实时交互与即兴追问，天然具备防范 LLM 代答的优势，能够有效检验学生对实验代码的真实理解深度。然而，传统人工口试面临严峻的规模化困境：以百人规模的 OS 实验班级为例，若按每名學生 20–30 分钟、配备双考官计算，完成一轮实验口试需要消耗数百小时的教师工时，在现有教学资源配置下几乎不可行。因此，如何在保持口试评估效度的同时，突破其可扩展性瓶颈，成为亟待解决的关键问题。

(3) 通用 AI 评估工具缺乏操作系统领域的专业适配性。现有基于 LLM 的自动出题与评分系统多面向通用知识问答或商业案例分析，其知识库与评估标准难以直接迁移至操作系统实验场景。OS 实验涉及大量底层概念、复杂的内核源码上下文以及开放性的实现路径，要求出题与评分系统具备领域知识增强能力。如何构建面向 ucore 和 rcore 教学操作系统的专业知识库，并设计基于检索增强生成（RAG）的自动出题机制，是本研究需要解决的核心技术问题之一。

(4) 单一 LLM 评分的主观性与不稳定性。开放式实验问题的评分具有高度主观性，单一 LLM 在评判时易受模型训练偏差、提示敏感性及上下文长度限制的影响，导致评分标准漂移与结果波动。现有研究虽已提出多模型协作评估的初步框架，但在评分离群值检测、置信度量化及自动重评机制方面仍缺乏系统性设计。如何构建一套鲁棒、可解释且具备自我修正能力的多维度评分体系，是保障 AI 口试评估效力的关键。

1.3.2 系统设计目标

针对上述研究问题，本研究旨在设计并实现一套面向操作系统课程实验的自动化 AI 口试系统，具体系统设计目标如下：

(1) 基于 RAG 知识增强的自动出题模块。系统应能够接收学生提交的实验代码，自动解析其修改的源码文件与关键函数，并基于构建好的 OS 领域 RAG 知识库检索相关上下文，生成具有专业性、针对性与层次感的口试问题。问题应覆盖代码实现细节、设计决策依据与底层概念理解三个维度，并通过互异性检查机制避免重复提问。同时，系统需支持 ucore 与 rcore 两种教学操作系统知识库的动态切换，以适应不同教学场景。

(2) 低延迟、高稳定性的实时语音口试交互系统。系统应集成自动语音识别 (ASR) 与文本转语音 (TTS) 技术, 构建基于 WebSocket 的全双工语音通信通道, 实现“语音播报题目—考生语音回答—实时转写与理解”的闭环交互。考试界面需支持代码片段的同步展示与语音播报的双模态呈现, 确保考生在回答涉及源码细节的问题时能够直观参照。系统还需具备考试状态机管理、超时处理、异常恢复及 OS 专用口试模式与普通调试模式的双模式适配能力。

(3) 多维度、可解释的多大语言模型协作智能评分模块。系统应设计三阶段评分流程: 第一阶段由多个异构 LLM 独立对考生回答进行多维度评分 (正确性、完整性、深度、代码理解程度) 并生成结构化分析报告; 第二阶段实施模型间交叉评审, 每个模型审阅其余模型的评分依据并修订自身评分; 第三阶段由主席模型综合所有评分与交叉评价生成最终成绩与反馈。此外, 系统需引入基于统计离群值检测的置信度检验机制, 对显著偏离的评分自动触发重评, 并将重评结果纳入最终裁决, 以提升评分的一致性与可信度。

第 2 章 相关技术

2.1 大语言模型技术

2.1.1 生成式预训练模型原理

当前大语言模型 (Large Language Models, LLMs) 的发展以 OpenAI 的 GPT 系列为代表, 其核心架构建立在 Vaswani 等人于 2017 年提出的 Transformer 之上。Radford 与 Narasimhan 在 2018 年提出的 Generative Pre-trained Transformer (GPT) 首次验证了“无监督预训练 + 有监督微调”的两阶段范式: 模型首先在大规模无标注文本上进行生成式预训练, 学习语言的通用表示; 随后针对具体下游任务进行微调, 仅需少量标注数据即可获得优异性能。GPT 采用仅解码器 (decoder-only) 的 Transformer 架构, 通过掩码自注意力机制实现从左到右的文本生成, 其 1.17 亿参数的规模在当时已刷新多项语言理解基准^[7]。

该范式在后续工作中被持续放大。Radford 等人于 2019 年发布的 GPT-2 将参数量扩展至 15 亿, 并引入 WebText 数据集进行无监督多任务学习, 验证了模型在不依赖下游微调的情况下通过提示完成零样本任务的能力^[8]。Brown 等人于 2020 年进一步将 GPT-3 扩展至 1750 亿参数, 系统性地证明了大规模语言模型具备少样本学习 (few-shot learning) 能力, 仅需在提示中提供少量示例即可适应新任务^[9]。Kaplan 等人提出的“Scaling Laws”则从理论上揭示了模型性能随参数量与数据量增长的幂律关系, 为后续大模型的规模化发展奠定了理论基础^[10]。

然而, 单纯扩大模型规模带来了安全性、价值观对齐与指令遵循等方面的挑战。Ouyang 等人于 2022 年提出的 InstructGPT 引入“基于人类反馈的强化学习”, 通过训练奖励模型对人类偏好进行建模, 并使用近端策略优化 (Proximal Policy Optimization, PPO) 算法对预训练模型进行微调, 显著提升了模型生成有用、诚实且无害回复的能力^[11]。RLHF 现已成为 GPT-4、Claude 等主流对话模型的标准后训练流程。

2.1.2 基于提示词工程的代码分析

提示词工程是指通过精心设计的输入模板引导 LLM 完成特定任务的技术。在代码分析场景中, 提示词工程的有效性体现在两个层面: 其一, 通过角色设定与任务描述, 引导模型以代码审查者或调试专家的身份进行推理; 其二, 通过上下文学习 (in-context learning) 在提示中嵌入代码片段、错误日志或相关文档, 使模型能够基于有限示例理解特定代码库的结构与语义。

代码分析任务对提示设计提出了特殊要求。由于程序代码具有严格的语法结构与执行语义，提示需要兼顾自然语言描述与形式化表示。研究表明，将代码的抽象语法树（AST）路径或控制流信息以结构化方式嵌入提示，可显著提升 LLM 在代码理解、漏洞检测与程序修复任务上的表现^[12]。此外，链式思维提示通过要求模型分步推理，能够有效改善复杂代码逻辑的分析质量^[13]。

2.1.3 多 Agent 协作机制

随着单一 LLM 在复杂任务上的局限性逐渐显现，研究者开始探索多 Agent 协作机制。Tran 等人于 2025 年发表的综述系统地梳理了 LLM 多 Agent 系统的协作维度，包括参与者、协作类型、通信结构与协调策略。在集中式结构中，一个中心 Agent 作为协调枢纽聚合多个 LLM 的响应，例如 LLM-Blender 通过成对排序融合不同模型的输出；在分层结构中，AgentVerse 等框架将 Agent 按功能分层组织，实现复杂任务的分解与协作求解^[14]。

多 Agent 协作在评估任务中的应用尤为关键。Liang 等人提出的 ChatEval 框架通过多 Agent 辩论机制，让多个 LLM 以评审者身份相互质疑与修正，从而提升开放式生成任务的评估质量^[15]。这一思想与本研究的多 LLM 协作评分的核心设计高度契合。

2.2 检索增强生成（RAG）技术

2.2.1 RAG 基本架构

检索增强生成（Retrieval-Augmented Generation, RAG）由 Lewis 等人于 2020 年提出，旨在通过外部知识检索弥补 LLM 在知识密集型任务中的固有缺陷^[16]。其核心架构包含两个模块：检索器负责从外部知识库中召回与查询语义相关的文档片段；生成器则将检索到的上下文与原始查询拼接后输入 LLM，引导模型生成基于事实的回复。该架构有效缓解了 LLM 的知识截止、参数化知识静态化及幻觉等问题^[17]。

2.2.2 向量数据库与语义检索

向量数据库是 RAG 系统的核心基础设施，负责存储文档的稠密向量表示并支持高效的近似最近邻（Approximate Nearest Neighbor, ANN）检索。ChromaDB 作为轻量级开源向量数据库，因其易于集成、支持内存检索与持久化存储等特性，在学术研究中被广泛采用。其底层通常基于 Hierarchical Navigable Small World（HNSW）算法实现子线性复杂度的语义相似性搜索。

语义检索的质量高度依赖嵌入模型的选择。Zhang 等人的研究表明，Qwen3-Embedding 系列通过大规模合成数据预训练与领域微调，在代码检索等特定领域任务上取得了显著的性能提升^[18]。此外，多项实证研究也证实，在垂直领域中对通用嵌入模型进行领域微调，可使检索准确率提升 5%–10% 甚至更高^[19]。在 RAG 流水线中，文档首先被切分为适当长度的文本块，经嵌入模型编码为向量后存入 ChromaDB；查询时通过余弦相似度计算召回最相关的 top-k 文档片段^[16]。

2.3 语音处理技术

2.3.1 自动语音识别 (ASR) 原理与开源方案

自动语音识别 (Automatic Speech Recognition, ASR) 旨在将音频信号转换为文本序列。现代 ASR 系统主要基于深度神经网络架构，包括基于 Transformer 的编码器-解码器模型与基于 Connectionist Temporal Classification (CTC) 的流式识别模型。在开源生态中，sherpa-onnx 项目提供了基于 ONNXRuntime 的高效推理框架，支持多种预训练模型的本地部署，包括 SenseVoice 等多语种语音识别模型^[20]。FunASR 则是阿里巴巴达摩院推出的开源语音识别工具包，集成了 Paraformer 等高性能模型，在中文语音识别任务上表现优异^[21]。这些开源方案为构建低延迟、高可用的语音考试系统提供了技术基础。

2.3.2 文本转语音 (TTS) 技术

文本转语音 (Text-to-Speech, TTS) 技术经历了从拼接合成到参数合成再到神经网络端到端合成的演进。当前主流的神经网络 TTS 模型 (如 VITS、Bark、GPT-SoVITS) 能够生成自然度接近真人的语音，并支持语速、语调与音色的调控。在考试场景中，TTS 不仅需要准确播报题目文本，还需处理技术术语 (如 sys_trace、trapframe) 的特殊发音，这对语音前端处理 (text normalization) 提出了额外要求。

2.3.3 实时语音流处理与 WebSocket 通信

实时语音交互要求系统具备低延迟的双向通信能力。WebSocket 协议通过建立持久化的全双工连接，避免了 HTTP 轮询带来的额外开销，成为语音流实时传输的标准方案。在考试场景中，前端通过 WebSocket 将音频流分段发送至后端 ASR 服务，后端完成识别后通过 TTS 合成回复音频并回传，形成“收听—识别—理解—合成—播放”的完整交互闭环。

2.4 多模型集成评估方法

2.4.1 单模型评估的局限性

使用单一 LLM 作为自动评估器虽然操作简便，但存在系统性偏差风险。Li 等人于 2024 年发表的综述系统梳理了 LLM 评估器面临的核心挑战，包括位置偏差、长度偏差、自我偏好偏差及提示敏感性^[22]。Koo 等人在 ACL2024 上发表的实证研究进一步量化了这些认知偏差：在 16 个不同规模的 LLM 评估器上，约 40% 的比较判断中出现了显著的自我偏好（即模型倾向于给自己的输出更高评分），且机器偏好与人类偏好的 Rank-Biased Overlap (RBO) 得分仅为 44%，表明现有 LLM 评估器与人类判断存在明显错位^[23]。

2.4.2 多模型投票与一致性检验方法

为缓解单模型评估的偏差，研究者提出了多模型协作评估（Multi-LLM Evaluation）框架。其核心思想是让多个异构 LLM 独立对同一输出进行评分，再通过投票或加权聚合得出最终结论。Chan 等人提出的 ChatEval 框架通过多 Agent 辩论机制，让不同模型相互评审并修正评分，显著提升了评估的一致性^[24]。在聚合策略上，除简单多数投票外，还可采用基于置信度的加权平均，或引入元评审模型对多模型输出进行更高层次的综合裁决。

评估一致性的量化通常采用 Krippendorff's α 系数，该指标能够处理不同评卷者数量不一致的情况，并适用于名义、序数与区间等多种数据类型。在 LLM 评估研究中， $\alpha > 0.8$ 通常被视为高度一致， $0.667 \leq \alpha < 0.8$ 为可接受范围^[25]。

2.4.3 离群值检测与鲁棒性评估

多模型评估中，个别模型可能因训练偏差、提示敏感性或领域知识不足而产生显著偏离群体共识的评分。识别并处理这类离群值是保障评估鲁棒性的关键环节。统计上常用的离群值检测方法包括基于四分位距（Interquartile Range, IQR）的 Tukey 围栏法与基于标准差的 Z-score 方法。IQR 方法通过计算评分分布的第一四分位数（Q1）与第三四分位数（Q3），将超出 $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ 范围的评分判定为离群值，该方法对非正态分布具有较好的鲁棒性。

在检测到离群评分后，系统可触发自动重评机制：要求偏离模型在补充上下文或修正提示后重新评分，并将重评结果与原始评分一并提交给元评审模型进行综合裁决。这一机制有效降低了单一模型异常输出对最终评估结果的干扰，提升了多模型评估体系的可靠性。

2.5 现有技术总结

综合上述分析，现有研究在以下四个维度取得了显著进展：在自动出题与代码评估方面，LLM 通过提示词工程显著提升了问题生成的质量与上下文关联性，但在“识别抄袭”与“测评理解深度”之间存在明显失衡；在多模型协作评估方面，研究者提出了独立评分、交叉评审与主席裁定的三阶段框架，初步验证了多模型协作在降低个体偏差上的有效性，但评分离群值检测、置信度量化与自动重评机制仍缺乏系统性设计；在语音交互集成方面，端到端 ASR 与 TTS 技术的成熟大幅降低了语音考官的构建门槛，然而现有系统多聚焦于通用知识问答或商业案例分析，尚未针对操作系统等底层系统类课程的实验特点进行专门适配；在 RAG 教育应用方面，外部知识检索有效缓解了 LLM 的知识幻觉问题，但 RAG 技术与语音口试系统的结合尚处于概念阶段，缺乏完整的工程实现与教学实验验证。

第 3 章 系统设计

3.1 需求分析

本研究面向的核心场景是操作系统课程实验的自动化口试评估。与一般在线考试系统不同，本系统并不以批量生成选择题或静态判题为主要目标，而是围绕学生提交的 OS 实验代码，构建一个能够发现代码变更、提出针对性问题、通过语音交互追问并最终形成可信评分报告的闭环。

3.1.1 功能需求

系统的功能需求可以分为五类。第一，代码提交与差异分析。考生需要提交修改前和修改后的实验代码压缩包，系统需从压缩包中提取 C、Rust、汇编、Makefile、Markdown 等与 OS 实验相关的文件，过滤 `target`、`build`、`.git` 等无关目录，并计算新增、删除、修改文件及变更行号。该需求对应项目中的 `CodeExtractor` 与 `DiffAnalyzer`，是后续出题的输入基础。

第二，自动出题。系统应根据实验要求、代码差异、关键变更类型以及课程知识库检索结果生成口试主干问题。问题不能停留在泛泛的概念回忆层面，而应直接关联学生修改过的代码片段，覆盖概念理解、实现细节、调试能力、优化思考和整体设计理解等类型。实际实现中，`questions/os_proposer.py` 将问题封装为 `ProposedQuestion`，每个问题包含题目文本、类别、出题理由以及若干 `code_snippets`，供前端展示和语音口试使用。

第三，实时语音口试。系统需要在前端展示问题和代码片段，同时以语音形式播报问题，并采集考生语音回答。考官不能仅按固定脚本念完题目，而应支持追问、重复、解释、提示和跳题等交互动作。项目中的 `OralExaminer` 通过状态机维护当前题目、追问深度、考试轮次和超时状态；`OSOralExamSession` 则将 OS 出题器生成的问题作为主线，在学生回答后调用大模型生成围绕当前问题的动态追问。

第四，自动评分与结果反馈。系统需要把口试过程中的问题和学生回答整理为问答对，提交给多模型评分委员会，输出每题分数、等级、置信度、主席评语以及整体理解程度、知识漏洞和学习建议。该功能由 `llm-council/backend/council.py` 的三阶段评分流程和 `server.py` 中的 `chairman_overall_assessment` 共同完成。

第五，过程记录与复盘。系统应保存考试会话、对话转写、代码片段引用、评

分结果和总体评估，供教师复核。当前原型采用内存型 `EvaluationStorage` 管理会话数据，并在 OS 出题阶段将生成的问题集保存为 JSON 与 Markdown 文件，便于调试和追踪。

3.1.2 非功能需求

在非功能需求方面，首先是实时性。语音口试要求系统在考官发问、考生作答、ASR 转写、LLM 追问和 TTS 播报之间保持较低延迟。系统采用 FastAPI 异步接口与 WebSocket 长连接，避免频繁轮询；耗时的 LLM 出题和语音识别任务通过异步任务或线程池执行，减少阻塞。

其次是可靠性。口试过程可能出现网络断开、考生长时间沉默、ASR 失败、TTS 生成失败或 LLM 响应格式异常等问题。系统通过心跳包检测连接状态，通过超时监控进行分级提醒，并在评分阶段设置解析失败回退逻辑。例如，当主席模型不可用时，评分模块可退化为使用教师模型平均分。

再次是可扩展性。OS 课程存在 `ucore` 与 `rcore` 两个不同的教学系统，知识库、代码语言和实验结构均不完全一致。因此系统在 RAG 工具层引入 `COLLECTION_MAP`，将 `ucore` 和 `rcore` 分别映射到独立的 ChromaDB collection；前端也提供课程类型选择，使同一套口试流程能够适配不同 OS 实验框架。

最后是可解释性。系统的评估结果必须能够向教师和学生说明依据。为此，出题模块保留检索工具调用、代码变更摘要和出题理由；评分模块保留各模型独立评分、同行评议排序、分数分布和主席裁定文本；前端则展示多维评分、置信度和具体反馈。

3.1.3 用户角色分析

系统主要涉及两类用户。考生负责上传代码、进入语音口试、查看或接收反馈。考生在口试过程中不仅回答主干问题，也可以通过“请重复”、“解释一下”、“给点提示”、“下一题”等指令控制对话节奏。教师或管理员负责配置课程类型、实验要求、题目数量、知识库内容和评分模型，并对低置信度结果进行抽查复核。由于本系统的目标是减轻助教口试负担，而非完全替代教师判断，因此教师仍保留对最终成绩的解释权和干预权。

3.2 系统架构设计

3.2.1 前后端分离架构

系统采用 React 前端与 FastAPI 后端分离的架构。前端位于 `frontend/src/App.tsx`, 提供 OS 实验口试入口、代码压缩包上传、课程类型选择、语音控制面板、对话记录展示和结果页。后端位于 `api/server.py`, 统一提供 HTTPAPI 与 WebSocket 入口。HTTP 接口用于启动考试、上传代码、生成问题、提交文字答案和查询结果; WebSocket 接口用于承载实时语音口试中的控制消息、音频二进制数据与考官响应。

该架构的优点在于将交互逻辑和评估逻辑分离。前端专注于录音、播放、状态展示和代码片段渲染; 后端专注于代码分析、RAG 出题、口试状态管理、ASR/TTS 和多模型评分。二者通过 JSON 消息约定交互, 使语音模块和评分模块可以独立演进。

3.2.2 核心模块划分

系统核心模块包括五部分。

(1) 代码差异分析模块。该模块接收修改前、修改后的 zip 文件, 提取源码文件并使用 `difflib.unified_diff` 生成统一 diff, 再记录新增文件、删除文件、修改文件和变更行号。OSExperimentAnalyzer 进一步根据关键词识别变更涉及的 OS 领域, 例如内存管理、进程调度、同步互斥、文件系统、中断和启动流程。

(2) RAG 增强出题模块。该模块由 QuestionProposer 负责。它先调用 LLM 分析代码变更并生成知识库检索计划, 再由 RAGToolKit 执行课程资料检索, 最后将实验要求、代码 diff、关键变更和检索结果一并输入 LLM, 生成带代码片段的口试问题。

(3) 语音口试模块。该模块由 `voice_oral_exam.py`、`oral_exam_engine.py` 和 `voice_service.py` 共同实现。它维护 WebSocket 会话、考试状态机、超时提醒、语音识别、语音合成和对话记录。OS 模式下, 系统以预生成问题为主线, 同时允许基于学生回答动态追问。

(4) 多模型评分模块。该模块基于 `llm-council` 子项目改造而来, 采用独立评分、同行评议、主席裁定三阶段机制。

(5) 结果管理与反馈模块。该模块由后端 `EvaluationStorage` 和前端结果页共同构成。后端保存考试状态、原始实验要求、问题集、口试记录和评分报告; 前端根据查询结果展示整体理解程度、置信度、主席反馈和后续建议。

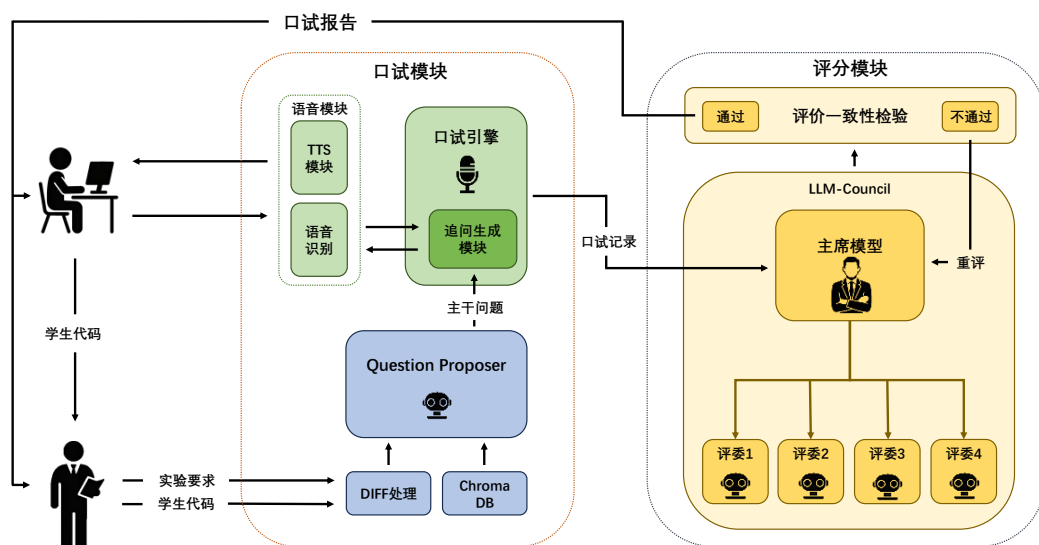


图 3.1 系统核心模块处理流程

3.2.3 技术栈选型与部署架构

后端采用 Python 与 FastAPI。FastAPI 支持异步请求处理和 WebSocket，适合构建实时口试服务。RAG 部分采用 ChromaDB 作为向量数据库，使用 Hugging Face Embeddings 加载本地嵌入模型，默认模型为 BAAI/bge-large-zh-v1.5，并提供 bge-small-zh-v1.5 回退机制。LLM 调用通过 Moonshot Kimi API 完成，评分委员会中配置了 kimi-k2.5、kimi-k2-0905-preview、kimi-k2-turbo-preview 和 kimi-k2-thinking 四个模型变体保证评价的丰富性。此部分也保留了接口供实际部署时选用不同的评分模型。

语音处理方面，当前实现选择 faster-whisper 作为本地 ASR 引擎，使用 CPUint8 推理以降低部署门槛；TTS 采用 edge-tts。前端采用 React、TypeScript、TailwindCSS 和浏览器 WebAudioAPI，录音侧使用 MediaRecorder，播放侧使用 AudioContext 解码后播放后端返回的 MP3 音频。

部署时，FastAPI 后端默认运行在 localhost:8000，前端通过 http://localhost:8000 调用 HTTP 与 WebSocket 接口。ChromaDB 数据持久化目录默认位于 /chroma，OS 出题结果保存到 questions/results。这种部署方式适合课程助教在本地或教学服务器上快速运行原型系统。

3.3 核心业务流程设计

系统的完整 OS 口试闭环如下：

第一步，教师或考生在前端填写实验要求，选择 ucore 或 rcore，并上传修改前、修改后两个代码 zip。前端通过 `POST/api/oral-exam/os-start` 将表单发送给后端。

第二步，后端提取代码并计算差异。`CodeExtractor.extract_from_zip` 返回路径到 `CodeFile` 的映射；`DiffAnalyzer.analyze` 生成 `CodeDiffReport`；`OSExperimentAnalyzer.identify_key_changes` 根据文件内容和函数变更识别关键 OS 修改点。

第三步，系统进行 RAG 增强出题。`QuestionProposer.generate_questions` 首先分析代码差异并规划知识库检索，然后执行检索，最后生成若干个 `ProposedQuestion`。这些问题既包含自然语言题干，也包含来自学生修改部分的代码片段。

第四步，后端创建 `OSOralExamSession`，将问题集注入 `OralExaminer`，并向前端返回 `evaluation_id` 和 `WebSocket` 地址。前端立即进入考试界面，连接 `WebSocket` 并发送 `start_exam` 指令。

第五步，语音口试开始。考官生成第一道主干问题，后端将题目文本、代码片段和音频分别发送给前端。考生点击录音按钮回答，前端将音频编码为 base64 后通过 `WebSocket` 发送给后端。后端调用 ASR 得到文本，再根据文本判断是否为快捷指令或正常答案。若为正常答案，系统将其记录到对话历史，并根据当前追问深度决定继续追问或进入下一题。

第六步，考试结束后，`OSOralExamSession.finish_exam` 从对话历史中抽取问答对，调用 `run_council_on_qa_pairs` 对每个问答对执行三阶段评分，再由主席模型生成整体评估。最终结果被写入会话存储，并通过 `WebSocket` 返回前端。

3.4 数据模型与接口设计

3.4.1 核心实体定义

系统中的核心实体包括：

(1) `CodeFile`。表示一个代码文件，包含路径、内容、语言类型和行数，并提供按行截取代码片段的方法。

(2) `FileDiff` 与 `CodeDiffReport`。前者表示单个文件的变更，包含旧

内容、新内容、统一 diff、旧行号和新行号；后者汇总代码包级别的新增、删除、修改和未变文件。

(3) `ProposedQuestion` 与 `QuestionSet`。`ProposedQuestion` 表示一道 OS 口试题, 包含 id、题目类别、题干、出题理由和代码片段; `QuestionSet` 表示一次实验生成的问题集合, 同时保存摘要和元数据, 例如 RAG 工具调用次数、上下文长度和知识库类型。

(4) `OralExamState` 与 `DialogueTurn`。`OralExamState` 保存口试状态, 包括原始实验要求、对话历史、当前追问深度、最大追问深度、考试轮次、超时阈值和等待状态; `DialogueTurn` 表示一次考官或学生发言, 包含角色、内容、时间戳和发言类型。

(5) `QuestionScoreDetail` 与 `OverallAssessment`。前者记录单个问答对的最终分数、等级、置信度、主席评语、教师模型评分和共识统计; 后者记录整场口试的理解程度、评估置信度、评估理由、知识漏洞和学习建议。

3.4.2 前后端关键 API 设计

与 OS 口试相关的主要接口如下:

`POST/api/oral-exam/os-start` 用于启动 OS 语音口试。请求字段包括 `experiment_requirement`、`before_zip`、`after_zip`、`student_id`、`num_questions` 和 `course`。返回值包括 `evaluation_id`、WebSocket 地址、课程类型、问题数量和语音配置。

`WebSocket/ws/oral-exam/{evaluation_id}` 用于实时口试。前端发送的消息包括 `start_exam`、`audio_data`、`text_data`、`interrupt`、`end_exam` 和心跳消息; 后端返回的消息包括 `examiner_response`、`audio_start`、二进制音频、`audio_end`、`transcription`、`listening`、`grading_started` 和 `exam_complete`。

`GET/api/evaluation/{evaluation_id}` 用于查询评估状态。当状态为 `completed` 时, 接口返回整体理解程度与置信度; 前端也可据此轮询评分完成状态。

第 4 章 RAG 知识库与自动出题模块

4.1 操作系统课程知识库构建

4.1.1 知识来源与数据预处理

操作系统实验口试的关键在于问题必须贴近课程语境。通用 LLM 虽然能够回答“什么是页表”、“什么是进程调度”等概念性问题，但很难稳定掌握特定教学操作系统中的函数命名、实验阶段、代码组织方式和教学要求。因此，本系统引入面向 OS 课程的 RAG 知识库，以降低模型幻觉并增强问题的课程相关性。

知识库的主要来源包括 ucore 与 rcore 实验指导书、SphinxHTML 课程文档、教学操作系统源码说明以及与实验章节相关的代码片段。chroma/extractor.py 中的 HTMLExtractor 用于从 HTML 文档中抽取结构化章节内容。它优先定位 article 或 div.content 主体区域，按 h1 至 h4 标题划分章节，同时保留段落、列表、代码块和提示框内容。每个章节被格式化为包含标题、正文和元数据的文档单元，其中元数据记录来源路径、标题层级、所属文档和字符数。

在 OS 代码上传阶段，系统还会对学生提交的 zip 文件做轻量级预处理。CodeExtractor 根据文件扩展名筛选.c、.h、.rs、.S、.ld、Makefile 等源码或配置文件，跳过构建产物和版本控制目录。这样既降低了分析输入的噪声，也避免将大量无关二进制文件送入 LLM 上下文。

4.1.2 文档切分与语义分块策略

RAG 检索的基本单元是文档块。系统在 KnowledgeBase.add_documents 中使用 RecursiveCharacterTextSplitter 对长章节进行切分，默认块大小为 500 字符，重叠长度为 50 字符，分隔符优先级依次为段落、换行、句号、分号、空格和单字符。选择 500 字符作为默认块长，是基于嵌入模型语义表示能力与 OS 概念完整性之间的权衡：块太短会割裂操作系统中紧密关联的概念链，导致检索召回的片段丢失关键上下文；块太长则会稀释单个块的主题集中度，降低余弦相似度匹配的精度，并大量占用 LLM 的输入窗口。500 字符大致对应 3–4 个自然段或一个完整的函数说明，能够在 OS 课程多数文档中保持单一主题的相对独立。50 字符的重叠长度约为块长的 10%，其设计意图是保障跨段落概念的语义连续性——例如当“页表项有效位”与“物理页号映射”被切分到相邻块时，重叠区域可确保二者在检索中至少有一个块同时包含部分关联信息，减

少边界信息丢失。

对代码片段而言，系统没有直接将整份学生代码纳入向量库，而是在考试时通过 diff 计算动态抽取变更上下文。这样设计有两点原因：一是学生代码属于每次考试的临时输入，不适合长期混入课程公共知识库；二是口试问题应聚焦学生修改部分，而非对整个项目进行泛化检索。课程知识库提供稳定背景知识，代码 diff 提供个体化上下文，二者在出题提示中融合。

4.1.3 向量嵌入模型选择与 ChromaDB 存储方案

系统使用 ChromaDB 作为向量数据库，并通过 `chromadb.PersistentClient` 将数据持久化到本地目录。嵌入模型默认采用 BAAI/bge-large-zh-v1.5，该模型适合中文技术文档的语义表示；如果本地缓存不可用，则尝试回退到 BAAI/bge-small-zh-v1.5。为提高部署稳定性，`chroma/database.py` 中显式设置 `HF_HUB_OFFLINE` 与 `TRANSFORMERS_OFFLINE`，并优先从本地 HuggingFace 缓存中查找模型 snapshot，避免考试时因网络波动导致知识库不可用。

每个 Chroma 文档块保存文本内容、向量表示和元数据。检索时，`KnowledgeBase.query` 调用 `similarity_search_with_score` 返回 top-k 结果，并将相似度转换为 `relevance_score`。出题器随后将检索结果格式化为包含来源、相关度和正文片段的文本，注入最终问题生成提示中。

4.1.4 课程适配机制

ucore 与 rcore 在语言、实验组织和知识点表达上存在明显差异。ucore 以 C 语言教学内核为主，常见考点包括物理页管理、页表、陷入处理和进程控制块；rcore 以 Rust 教学内核为主，常见考点涉及所有权语义下的内核抽象、SV39 页表、任务上下文切换和系统调用实现。系统通过 `RAGToolKit.COLLECTION_MAP` 将课程类型映射到不同 collection：ucore 对应 `ucore_2025s`，rcore 对应 `rcore_2025s`。前端在 OS 口试入口中提供课程选择，后端据此初始化或复用对应的 `OSQuestionProposer`。

这种双知识库机制避免了不同课程资料相互污染。例如，当学生提交 rcore 第三章任务切换实验时，系统应检索 Rust 版任务控制块和上下文切换资料，而不是召回 ucore 中的 C 结构体定义。课程级 collection 切换为后续扩展到其他系统类课程提供了统一模式。

4.2 基于 RAG 的知识增强出题

4.2.1 检索策略设计

本系统采用先规划、再检索、后生成的三阶段出题流程，而不是简单地用实验要求直接检索知识库。

第一阶段是检索规划。`QuestionProposer._analyze_and_plan_retrieval` 将实验要求、代码变更摘要、关键变更点和前两个修改文件的 diff 片段输入 LLM，要求其判断需要检索哪些课程知识。系统向模型暴露四种工具：`search_course_material` 用于通用资料检索，`search_os_concept` 用于概念检索，`search_by_lab` 用于按实验章节检索，`search_by_code_symbol` 用于按函数、结构体或符号名检索。模型必须输出 `<tool_calls>` 包裹的 JSON 数组，系统解析后得到工具调用计划。

第二阶段是执行检索。`RAGToolKit.execute` 根据工具名调用对应检索函数，并从 ChromaDB 返回若干课程资料片段。检索查询通常包含 OS 类型、概念名和实现语境，例如 `rcore` 页表原理实现或 `PageTable` 内存管理。这种由 LLM 规划检索词的方式比固定关键词搜索更灵活，能够从代码变更中归纳潜在考点。

第三阶段是问题生成。系统将实验要求、课程知识库检索结果、代码变更摘要、关键变更点和详细 diff 合并为最终上下文，要求 LLM 生成指定数量的面试问题。为了控制上下文窗口，`QuestionProposer.MAX_CONTEXT_LENGTH` 设为 12000 字符，过长内容会被截断。

之所以不采用“直接检索”的简单策略，是因为 OS 实验的代码变更具有高度领域特异性，而通用查询，如“内存管理”，会召回大量无关内容，导致 LLM 上下文被噪声淹没。让 LLM 先分析代码 diff 并规划检索，实质上是将“理解学生做了什么”作为“检索课程知识”的前置条件，使检索查询从静态关键词升级为动态语义描述，从而显著提升召回精度。

4.2.2 提示模板构建

出题提示模板强调六个原则：深度优先于广度、原理结合实践、能够区分真正理解与照搬代码、聚焦变更、循序渐进、结合课程知识。系统要求问题类型覆盖 `concept`、`implementation`、`debugging`、`optimization` 和 `understanding`，并以固定 Markdown 格式输出，便于后续解析。

为了适配语音口试，提示还要求每个问题必须具体、明确，避免一次提出多个不相关问题。由于 OS 实验问题经常依赖代码细节，提示中强制要求问题附带

代码片段，并用 `<code>...</code>` 标记包裹。后端解析问题时，会将代码片段从题干中剥离并保存到 `ProposedQuestion.code_snippets`，前端则将其渲染为独立代码块。

4.2.3 代码片段关联机制

代码片段关联是本系统区别于普通自动出题系统的关键设计。系统在最终上下文中优先提供修改文件的统一 diff，而非完整代码仓库。LLM 生成问题时被要求只引用学生实验代码修改部分，不得编造不存在的代码。解析阶段通过正则表达式提取 `<code>...</code>` 内容，形成题目文本与代码片段的结构化绑定。

在口试过程中，`OralExaminer._format_os_question` 会将题干和代码片段重新组合为考官话语。后端在发送前调用 `extract_code_snippets` 将 `<code>...</code>` 块替换为 [代码片段] 占位符，并把代码片段以数组形式随 `WebSocketJSON` 一起返回。前端 `ExaminerMessage` 组件根据占位符位置插入代码块，从而实现“语音播报题干，屏幕展示代码”的双模态呈现。TTS 侧不会逐字朗读代码，而是将占位符替换为“请看屏幕上的代码片段”，避免技术符号朗读造成理解困难。

4.3 出题质量控制

4.3.1 问题互异性检查机制

在自动出题场景中，LLM 容易围绕同一概念重复提问，例如连续生成多个“请解释页表是什么”的问题。为降低重复性，系统从提示和解析两层进行控制。在提示层，系统要求问题类型多样，并明确从具体实现、设计决策、边界条件和优化思考等角度循序渐进。由于最终上下文中包含关键变更点列表，模型可以围绕不同修改文件或不同函数构造问题。

在结构层，`QuestionSet` 保存每道题的类别和出题理由，便于后续对题目集合进行人工或程序化检查。当前原型尚未实现基于句向量相似度的自动去重，但其数据结构已经为该能力预留了位置。后续可对生成题目再次嵌入，若两题语义相似度超过阈值，则触发重生成或替换。

4.3.2 问题质量过滤

系统对 LLM 输出进行了基础质量过滤。首先，若响应中包含 API 错误标记，解析器直接返回空问题集；其次，主解析器要求问题符合预设格式，且题干长度大于最小阈值；再次，对于代码块，系统从题干中单独抽取并保存，避免 Markdown

混杂影响前端展示。若主解析失败，系统进入 fallback 解析逻辑，尝试从编号行中恢复问题。

此外，后端在返回题目前还会调用 `sanitize_llm_output` 和 `escape_html_in_code` 清理代码块语法与 HTML 特殊字符，减少前端渲染风险。对于 OS 口试而言，问题质量最终还体现在能否围绕学生回答动态追问。即便主干问题存在表述不够充分的情况，`_os_follow_up` 仍会结合当前问题、代码片段和学生回答生成更具体的追问，从交互层面补强考察深度。

第 5 章 多 LLM 协作评分模块

5.1 评分框架总体设计

5.1.1 多阶段协作思想

OS 口试的评分对象是开放式自然语言回答。学生可能使用不同表达方式解释同一段代码，也可能给出部分正确但不完整的答案。若仅依赖单一 LLM 直接打分，评分结果容易受到提示敏感性、模型偏好和领域知识覆盖不足的影响。为提升评分稳定性，本系统借鉴 LLM-Council 的思想，将评分过程拆分为独立评分、同行评议和主席裁定三个阶段。

选择三阶段而非简单投票或算术平均，是基于评估质量链路的系统性设计：独立评分阶段确保各模型在不受他人影响的情况下保留多样性判断，这是发现“边缘案例”的基础；同行评议阶段通过模型间的相互审视过滤个体偏见，实现“去偏差”；主席裁定阶段则由单一权威模型综合所有信息进行最终裁决，避免多模型简单聚合时可能出现的“中庸化”问题（即向平均分靠拢而丢失关键细节判断）。这一链路模拟了学术评审中“独立审稿，同行讨论，主编决策”的成熟流程。

实际实现位于 `llm-council/backend/council.py`。系统配置四个教师模型作为评分成员：`kimi-k2.5`、`kimi-k2-0905-preview`、`kimi-k2-turbo-preview` 和 `kimi-k2-thinking`。主席模型默认为 `kimi-k2.5`。每个口试问答对都会经过完整委员会流程，最终输出单题分数、等级和评语。整场口试再由 `chairman_overall_assessment` 汇总所有单题结果，形成对学生 OS 实验理解程度的整体判断。

5.1.2 评分维度定义

当前代码中的评分提示采用 10 分制，包含四个维度：知识准确性 4 分、内容完整性 3 分、逻辑与结构 2 分、表达与规范 1 分。结合 OS 口试场景，这四个维度可进一步解释如下。

知识准确性对应学生对 OS 原理和代码语义的判断是否正确。例如，在回答页表遍历问题时，学生是否能正确说明多级页表索引、页表项有效位和物理页号之间的关系。内容完整性对应回答是否覆盖关键点，例如是否同时说明正常路径、异常路径和边界条件。逻辑与结构对应学生是否能把概念、代码行为和设计意图连接起来，而不是堆砌术语。表达与规范则关注术语使用是否准确、回答是否可理解。

5.2 第一阶段：独立评分与报告生成

5.2.1 多 LLM 并行评分架构

第一阶段由 `stage1_teacher_scoring` 实现。系统将题目和学生答案封装为评分提示,并通过 `query_models_parallel` 并行请求所有教师模型。并行评分有两个优点:一是减少等待时间,二是保证各模型在看到他人意见前独立判断,从而保留多模型意见差异。

每个模型返回后,系统调用 `parse_score_from_text` 从文本中解析 0 至 10 的分数。每个教师模型的输出被保存为 `model`、`raw_response`、`score` 和 `feedback` 四个字段,其中 `feedback` 保留完整评语,供后续同行评议和主席裁定使用。

5.2.2 评分提示工程与约束设计

第一阶段提示词要求模型以“资深教育评估专家兼学科教师”的角色评分,并明确给出分值分配、各档标准和输出格式。固定输出格式是工程实现中的关键约束,因为后续阶段需要自动解析分数并生成统计结果。如果模型只给出自然语言评价而缺少可解析分数,系统将难以计算平均分、分歧程度和最终等级。

为避免评分过于笼统,提示要求评语包含优点、不足和改进建议。对于 OS 口试而言,这些反馈可以直接指出学生未掌握的代码或概念,例如“不清楚 `trap` 上下文保存顺序”、“未说明页表项无效时的异常处理”、“混淆了任务切换和进程调度”。这种结构化反馈为最终整体评估中的知识漏洞归纳提供了材料。

5.2.3 结构化评分报告生成

第一阶段输出不是最终成绩,而是多个独立评分报告。每份报告包含分数、维度分解和详细评语。后端在 `QuestionScoreDetail.teacher_scores` 中记录各模型名称和分数,在 `chairman_feedback` 中记录第三阶段主席评语。虽然当前前端展示较为简化,但后端数据已经包含完整评分链条,教师可以通过接口获取更细粒度的评分依据。

5.3 第二阶段：交叉评审与一致性检验

5.3.1 模型间交叉评审机制

第二阶段由 `stage2_peer_review` 实现。系统首先将第一阶段评分结果匿名化,分别标记为“评分 A”、“评分 B”等,并隐藏模型身份。随后,所有教师

模型再次并行调用，对这些匿名评分方案进行评审，评价其准确性、建设性、标准一致性和公平性，并给出从最佳到最差的排名。

匿名化设计并非简单的技术处理，而是基于社会心理学中的“盲审原理”与 LLM 评估研究的实证发现。Koo 等人的实验表明^[23]，LLM 评估器存在显著的自偏好偏差，即模型倾向于给自己的输出更高评分。若让模型在知道评分方案来源的情况下进行评审，这种偏差会被进一步放大。通过匿名化，模型被迫从“评分方案本身的质量”而非“模型身份”角度进行判断，促使评审聚焦于评分依据的充分性与标准的公平性。虽然当前四个评分成员均来自 Kimi 系列不同变体，异构性不如跨厂商模型明显，但匿名同行评议仍能降低模型对自身输出或特定版本号的隐性偏好。parse_ranking_from_text 会从输出中解析“评分 A”、“评分 B”等排名标签，供共识统计使用。

5.3.2 评分一致性度量方法

系统当前采用分数分布统计和排序投票来衡量一致性。calculate_scoring_consensus 计算参与教师数、最低分、最高分、平均分、中位数和标准差，并根据最高分与最低分的差值划分一致性等级：差值不超过 1.5 为高一致性，不超过 3 为中一致性，超过 3 为低一致性。同时，系统统计第二阶段中每个评分方案被评为最佳的次数，以判断哪些教师评分更受同行认可。

这种一致性度量方式简单、直观，适合原型系统快速落地。与 Krippendorff's α 或 Kappa 系数相比，它对样本规模要求更低，也更容易向教师解释。

5.3.3 置信度初步计算

置信度由第三阶段根据第一阶段分数差异自动给出。若教师间分数差异小于 1.5 分，系统标记为“高”置信度；差异在 1.5 至 3 分之间，标记为“中”；差异大于 3 分，标记为“低”。这一置信度不表示答案正确概率，而表示委员会内部评分意见的一致程度。

在 OS 口试中，低置信度通常意味着两类情况：一是学生回答本身模糊，部分模型认为其理解尚可，部分模型认为其缺少关键细节；二是题目或评分标准存在歧义，导致模型关注点不同。因此，低置信度结果应优先进入教师复核队列。

5.4 第三阶段：综合裁决与结果生成

5.4.1 元评审模型设计

第三阶段由 `stage3_chairman_final_grade` 实现。主席模型接收原始题目、学生答案、各教师模型评分、平均分、分数差异和同行评议摘要。提示中明确主席拥有最终裁定权，需要审查评分分歧、权衡同行意见并给出 0 至 10 分的最终成绩。

主席模型的输出格式包括最终得分、置信度、评定等级、综合评价、主要优点、关键不足、得分依据和学习建议。系统通过 `parse_final_score` 解析最终得分，并通过 `score_to_grade` 转换为等级，例如 A+、A、B+、B、C 和 D。

5.4.2 融合策略

本设计实现中的融合策略是统计信息辅助下的主席裁定。也就是说，系统并不简单取平均分，而是将平均分、分数范围、教师评语和同行排名全部提供给主席模型，由其给出最终判断。如果主席模型调用失败，系统才回退为教师平均分。

这种策略更适合开放式口试评分。平均分能反映总体倾向，但无法解释为什么某个严厉评分或宽松评分更合理。主席模型可以综合比较各方依据，例如在学生答案遗漏关键概念时倾向于较低评分，在学生表述不流畅但核心理解正确时避免过度扣分。

5.4.3 最终评分报告与评语生成

单题评分完成后，`run_grading_council` 返回第一阶段结果、第二阶段结果、第三阶段结果和元数据。OS 语音口试结束时，`OSOralExamSession.finish_exam` 会从对话历史抽取问答对，对每个问答对执行评分，并将结果整理为 `exam_scores`。随后，`chairman_overall_assessment` 汇总所有单题结果，生成整场考试的整体理解程度、知识漏洞和学习建议。

整体评估提示中包含实验要求、代码变更摘要、各深度测试问题的得分和反馈摘要。主席模型需要输出 JSON，包括 `understanding_level`、`confidence`、`reasoning`、`knowledge_gaps` 和 `recommendations`。这种设计使最终报告不只是分数列表，而是面向教学改进的诊断性反馈。

5.5 离群值检测与重评机制

5.5.1 统计离群值识别方法

多模型评分机制虽然能够缓解单模型偏差，但仍可能出现个别模型给出明显偏离群体判断的分数。为避免异常评分直接影响主席裁定，本系统在第二阶段同行评议结束后加入离群值检测流程，由 `detect_reevaluation_triggers` 对第一阶段评分进行统计分析，并输出需要重评的模型集合及触发原因。

系统同时采用 IQR、Z-score 和大分差中位数偏离三类方法。IQR 方法中 1.5 倍系数是在探索性数据分析中被广泛验证为平衡敏感性与鲁棒性的标准选择；对于近似正态分布，该阈值约对应均值 ± 2.7 个标准差，可覆盖 99.3% 的正常数据。Z-score 阈值设为 2.0，对应约 95% 置信水平下的双侧检验，适合小样本下的显著偏离识别。3 分分差阈值与 5.3.3 节置信度阈值保持一致，形成“低置信度，离群检测，重评触发”的连贯逻辑。中位数偏离 1.5 分的附加条件则用于防范 IQR 在小样本中过于保守的问题——当四个模型中有三个评分集中而一个极端偏离时，即使该分数未超出 $1.5 \times IQR$ ，中位数偏离条件仍能捕获异常。

除数值统计外，系统还利用第二阶段的同行评议结果。如果某个匿名评分方案在多个评审模型的排序中持续处于末位，说明其评分依据可能被其他模型认为不充分或不公正。系统统计每个评分方案被排在末位的次数，当次数达到阈值时，同样触发重评。最终，触发报告会记录 IQR 边界、Z-score、分数范围、中位数偏离、同行末位票数和具体触发模型，供后续主席裁定追溯。

5.5.2 动态重评触发条件与流程

动态重评流程由 `run_targeted_reevaluation` 实现。系统不会对所有模型重复评分，而只对被离群检测或同行末位机制标记的模型发起定向重评，以控制额外 API 调用成本。每个被触发的模型会收到一份重评提示，提示中包含原始题目、学生答案、该模型自己的原始分数和评语、触发重评的具体原因、其他匿名评分摘要以及同行评议摘要。

重评提示要求模型重新审阅题目和学生答案，并明确说明是否修正原评分。如果模型认为原评分合理，可以维持原分；如果发现原评分过严、过松或遗漏关键证据，则给出修正后的分数和理由。系统通过扩展后的 `parse_score_from_text` 解析“** 重评结果 **”字段，并保存 `original_score`、`revised_score`、`score_delta`、`trigger_reasons` 和完整重评文本。

在完整评分流程中，重评发生在第二阶段同行评议之后、第三阶段主席裁定之前。也就是说，主席模型看到的不再只是初始评分和同行排序，还包括自动重

评记录。这样既保留了多模型独立评分的原始判断，又允许模型在发现异常后进行自我修正。

5.5.3 重评结果融合策略

重评结果不直接覆盖第一阶段评分，而是作为独立证据提交给主席模型。这样设计主要是为了避免模型在看到其他评分后产生简单从众行为。如果重评文本能够指出原评分遗漏的关键事实，例如忽略了学生对异常路径的解释，主席模型可以提高重评分的参考权重；如果重评只是无理地向平均分靠拢，主席模型则可维持对原始独立评分的关注。

在代码实现上，`stage3_chairman_final_grade` 增加了可选的 `reevaluation_results` 参数，并在主席提示词中加入“自动重评记录”部分。主席的裁定任务也被扩展为：如存在重评记录，不要机械覆盖原始评分，而应判断重评是否基于新的证据或更严谨的标准，再决定其在最终分数中的权重。最终，重评触发报告和重评结果会被写入 `consensus_stats.reevaluation`，同时通过 `QuestionScoreDetail.reevaluation` 返回给前端或教师复核工具。该机制使系统形成“独立评分—同行评议—异常重评—主席裁定”的闭环，提高了多模型评分在开放式 OS 口试场景下的鲁棒性和可解释性。

5.6 评分可解释性设计

5.6.1 评分依据追溯

系统从三个层面保留评分依据。第一，题目层面保存出题理由、代码片段和 RAG 检索元数据，使教师能够追溯提问模型为什么问这个问题。第二，回答层面保存完整口试对话，包括考官主干问题、追问、学生转写文本和超时记录。第三，评分层面保存教师模型评分、同行评议、共识统计和主席裁定。通过这些数据，教师可以检查评分是否基于学生实际回答，而不是基于原始代码或模型推测。

5.6.2 多维度展示

前端当前主要展示整体评估结论与置信度，后端则已经返回每题分数和教师模型分数。后续可在结果页加入多维度雷达图，将知识准确性、内容完整性、逻辑结构和表达规范可视化；也可按 OS 知识点聚合低分问题，展示学生在内存管理、进程调度、中断处理等方面的薄弱程度。该设计能够将 AI 评分结果转化为更具教学价值的学习诊断。

第 6 章 语音口试模块设计与实现

6.1 语音交互架构

6.1.1 实时语音流处理架构

语音口试模块采用 WebSocket 全双工通信。前端连接 `/ws/oral-exam/{evaluation_id}` 后，发送 `start_exam` 消息启动考试。后端生成考官问题后，先发送 JSON 格式的 `examiner_response`，其中包含题目文本、代码片段、问题类型、追问深度和轮次；随后发送 `audio_start`、二进制 MP3 音频和 `audio_end`，前端据此播放考官语音。

考生回答时，前端使用浏览器 `MediaRecorder` 采集 `audio/webm;codecs=opus` 音频，每次录音最长 30 秒。录音结束后，前端将音频段转为 `base64`，并通过 `audio_data` 消息发送给后端。后端调用 `VoiceService.speech_to_text` 完成 ASR，再将转写结果通过 `transcription` 消息返回前端，同时把文本交给口试引擎处理。若考生使用快捷按钮，前端直接发送 `text_data`，跳过 ASR。

选择 WebSocket 而非 HTTP 轮询，是因为语音交互要求双向低延迟数据流：前端需持续上传音频，后端需实时下发控制消息与合成音频。HTTP 轮询的往返延迟通常在 300–500ms 以上，且每次请求需重建 TCP 连接与 TLS 握手，无法满足口试中频繁音频传输的实时性要求。WebSocket 建立一次连接后可持续复用，头部开销极小，且天然支持二进制帧传输，适合承载 MP3 音频数据。

6.1.2 语音状态机设计

系统的口试状态机围绕等待、收听、识别、思考、合成、播放循环展开。前端层面维护 `phase`、`connectionStatus`、`isExaminerSpeaking`、`isRecording`、`isProcessing`、`isWaitingForAnswer` 等状态；后端层面由 `VoiceOralExamSession` 和 `OralExamState` 维护会话激活状态、当前任务、是否正在说话、等待回答状态、追问深度和超时等级。

一次典型交互如下：后端生成问题时发送 `examiner_typing`，前端显示“考官正在准备问题”；问题生成后发送 `examiner_response`，前端展示文本和代码；后端调用 TTS 并发送音频，前端播放期间禁用录音；播放结束后进入 `listening` 状态，前端启用录音按钮；考生回答后进入 `processing` 状态，

后端识别并生成追问或下一题；若所有问题结束，后端发送 `exam_complete` 并进入评分阶段。

为支持真实口试中的打断行为，前端在考官说话时若考生点击录音，会先停止当前播放并向后端发送 `interrupt`。后端收到后取消当前任务、停止超时监控并返回中断确认。该机制使对话不必严格遵循“考官完全说完后才能回应”的僵硬模式。

6.2 语音识别与合成集成

6.2.1 ASR 模块集成

6.2.1.1 引擎选型与双模式架构

实际原型选用腾讯云自动语音识别（ASR）服务作为语音转写引擎，通过官方 Python SDK 接入其“一句话识别”与“录音文件识别”两条通道。选择云端 ASR 而非本地 `faster-whisper` 等开源方案，主要基于识别准确率与运维成本的权衡：操作系统口试涉及大量技术术语，云端 ASR 在中文连续语音识别与专业词汇建模上显著优于轻量级本地模型；同时，云端方案免去了本地 GPU/CPU 推理环境的维护，适合助教在教学服务器上快速部署，且腾讯云教育场景下网络延迟通常可控。

6.2.1.2 双路径识别策略

系统根据音频文件大小自动选择识别路径，以平衡实时性与准确率。若预处理后的 WAV 文件不超过 5MB，直接调用 `SentenceRecognition` 接口同步识别。音频数据经 Base64 编码后随请求体发送，SDK 在单次 HTTPS 往返内返回完整文本，平均延迟 300–800ms，满足口试实时交互需求。若音频超过 5 MB，则先通过腾讯云 COS SDK 上传至对象存储桶，获取临时预签名 URL；随后调用 `CreateRecTask` 创建异步识别任务，引擎在云端完成整段转写。后端通过 `DescribeTaskStatus` 以 2 秒为周期轮询任务状态，任务完成后，还原为纯文本返回前端。

6.2.1.3 异步封装与事件循环保护

腾讯云 Python SDK 的调用方式为同步阻塞式 I/O。为避免其等待时间占用 FastAPI 的异步事件循环、导致 WebSocket 心跳或超时监控等并发任务被挂起，系统通过 `asyncio.get_event_loop().run_in_executor` 将所有 SDK 调用（包括预处理、一句话识别、COS 上传、任务创建与轮询）委托给线程池执行。这种“异步接口 + 同步后端”的桥接模式，既保留了前端非阻塞体验，又兼

容了现有云 SDK 的同步设计。

6.2.2 TTS 模块集成与音色选择

TTS 模块基于 `edge-tts` 实现，默认音色为 `zh-CN-XiaoxiaoNeural`，默认语速为-10%。当考生要求重复问题时，系统将语速进一步降低到-25%，提升可听性。`VoiceService.text_to_speech` 会先调用 `_clean_text` 清理 Markdown 标记、行内代码、公式、链接和过长文本，再生成 MP3 文件并返回音频字节。

对于 OS 口试，TTS 的关键并非逐字朗读代码，而是与屏幕代码展示协同。后端在 `send_audio` 中将 [代码片段] 替换为“请看屏幕上的代码片段”，避免把 `fn`、`usize`、`trapframe` 等符号朗读成难以理解的音节。代码细节由前端代码块呈现，考官语音则负责引导学生关注代码片段中的具体问题。

6.2.3 音频数据预处理与降噪

前端采集麦克风时开启 `echoCancellation` 和 `noiseSuppression`，并请求单声道音频。后端转换音频时统一采样率和声道，ASR 推理时启用 VAD 过滤，以减少静音段和环境噪声的干扰。

6.3 口试考试流程实现

6.3.1 考试界面与状态同步

前端 OS 口试界面由左侧语音控制面板和右侧对话记录面板组成。左侧展示连接状态、考官是否正在说话、等待回答计时、录音按钮和快捷指令；右侧展示考官问题、代码片段、学生回答和系统提示。WebSocket 消息驱动界面状态变化，例如 `audio_start` 使录音按钮禁用，`listening` 使录音按钮启用，`processing` 显示“识别中”，`exam_complete` 切换到评分阶段。这种状态同步减少了考生误操作。例如，考官播放语音时录音按钮处于禁用或打断逻辑状态，避免播放音频被麦克风采集造成回声；ASR 处理中也禁止再次录音，避免多个音频任务交错。

6.3.2 题目语音播报与代码展示

OS 模式下，`OralExaminer` 不再完全依赖动态 LLM 出题，而是优先使用 `os_proposer` 预生成的问题集。第一题由 `_build_os_initial` 生成，并以“同学你好，我们开始口试”作为开场；后续题目由 `_build_os_next_topic` 逐题切换。每道题的类别会以 `[CONCEPT]`、`[IMPLEMENTATION]` 等形式保

留在文本中，方便前端和教师区分考察目标。

当问题包含代码片段时，后端将代码与文本分离发送。前端 CodeBlock 组件提供简单语法高亮和复制功能，支持 Python、JavaScript、Java、C/C++ 等语言的初步识别，以满足“让考生看到被追问代码”的核心需求。

6.3.3 考生语音回答采集与转写

考生点击录音按钮开始回答，再次点击结束录音。前端将录音数据发送给后端后，立即进入识别状态。后端返回转写文本后，前端将其作为学生发言加入对话记录。随后，`OralExaminer.process_student_input` 判断该文本是否包含快捷指令：若为“请重复”，则重述当前问题；若为“解释一下”，则用更简单语言解释；若为“给点提示”，则给出不直接泄露答案的提示；若为“下一题”或“我答完了”，则进入下一主干问题或结束考试。

若文本不是指令，则系统将其作为正式回答记录，并增加当前追问深度。OS 模式下，`_os_follow_up` 会基于当前主干问题、代码片段和学生回答生成一条简短追问，以进一步检测理解深度。当追问深度达到 `max_depth`，系统切换到下一题，避免在单个问题上无限追问。

6.3.4 超时处理与异常恢复机制

后端 `OralExamState` 设置了三档静默阈值，默认分别为 120、180 和 240 秒；前端也显示等待回答计时条。若考生长时间没有回答，后端超时监控会触发分级提醒：第一档提醒考生不用紧张，可请求解释；第二档建议请求提示或跳过；第三档则根据 `timeout_strategy` 决定结束或跳过。当前前端启动时使用 `prompt` 策略，即主要进行提醒而不立即强制结束。

异常恢复方面，WebSocket 会话包含心跳机制，每 5 秒发送 `ping` 以检测连接；若连接断开或已有活跃连接，后端会拒绝重复连接并清理任务。TTS 和 ASR 失败时，后端通过 `error` 消息告知前端，前端保留当前考试界面，允许考生重试或结束考试。考试结束阶段，即使评分过程耗时较长，系统也会先发送 `grading_started`，让前端进入等待结果状态，避免用户误以为连接已中断。

综上，语音口试模块通过 WebSocket、ASR、TTS、状态机和代码展示的组合，实现了面向 OS 实验的实时交互式考核。其核心价值不在于把文字题机械地读出来，而在于把学生代码、课程知识、语音回答和动态追问纳入同一个闭环，使系统能够更接近人工助教的口试行为。

6.4 端到端响应延迟特征分析

6.4.1 延迟分解与测量方法

为量化语音口试系统的实时交互性能，本研究对端到端响应延迟进行了系统性分解与测量。端到端延迟定义为：从考生结束语音回答（前端停止录音）到考官语音开始播放（前端收到 `audio_start` 消息）之间的完整时间间隔。该间隔可细分为五个串行阶段：（1）前端音频编码与 WebSocket 上行传输；（2）后端 ASR 语音识别处理；（3）LLM 追问生成；（4）TTS 语音合成；（5）音频下行传输与前端缓冲播放。

6.4.2 延迟测量结果

阶段分解揭示了关键的性能瓶颈。处理间隙（Other）以 40.3% 的占比成为最大延迟来源，该部分包含音频预处理、线程池调度等待、WebSocket 消息序列化与反序列化、以及阶段间的异步协程切换开销。值得注意的是，LLM 生成阶段虽仅占 36.2%，但是如果不进行追问而命中主干问题缓存，则此阶段不产生开销。TTS 合成阶段表现相对稳定，均值 2.38 秒，占比 19.7%。ASR 识别延迟最低，均值仅 464 ms，占比 3.8%，这得益于腾讯云一句话识别 API 的高效响应。

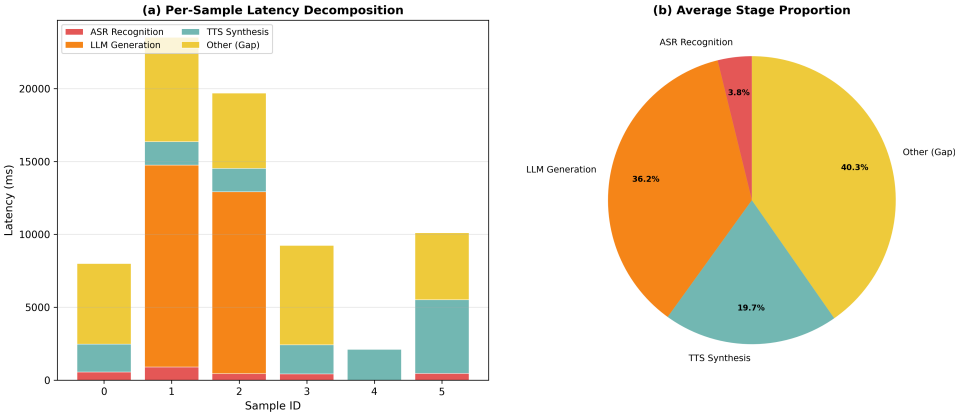


图 6.1 端到端响应延迟阶段分解

6.4.3 延迟分析

总体端到端延迟平均在 12 秒左右，在可接受的范围内。在考试过程中，考生会感受到一定程度的停顿。但是这样的停顿也有助于学生整理思路和调整心态，准备回答下一个问题。

为进一步优化端到端延迟，未来可以采用响应更快的追问模型，或者利用历史考试数据，预先生成追问问题缓存集。进一步降低追问相应缓存后，整体端到端延迟有望控制在 10 秒以下。

第 7 章 系统测试与实验评估

7.1 测试环境与数据集构建

7.1.1 测试环境

考试系统在 wsl 中进行了部署、功能测试以及自动化测试。所使用的大模型服务均由 KIMI API 提供。语音合成 (TTS) 采用微软 Azure 语音服务的 edge-tts 开源封装；语音识别 (ASR) 采用腾讯云一句话识别与录音文件识别服务。详细使用模型列表见附录 A。

7.1.2 测试数据集构建

本研究的测试数据集来源于清华大学操作系统课程实验的标准样例代码与实验指导文档。数据集内，每一章节均对应一份实验要求文本与两份代码快照压缩包，分别代表完成前和完成后的实验代码。根据两份提交代码的差异，以及实验要求，出题模块检索对应的 RAG 库，调用大模型完成出题。在后续的考试过程中，大模型根据学生回答进行追问。

7.2 RAG 功能测试

7.2.1 RAG 延迟测试

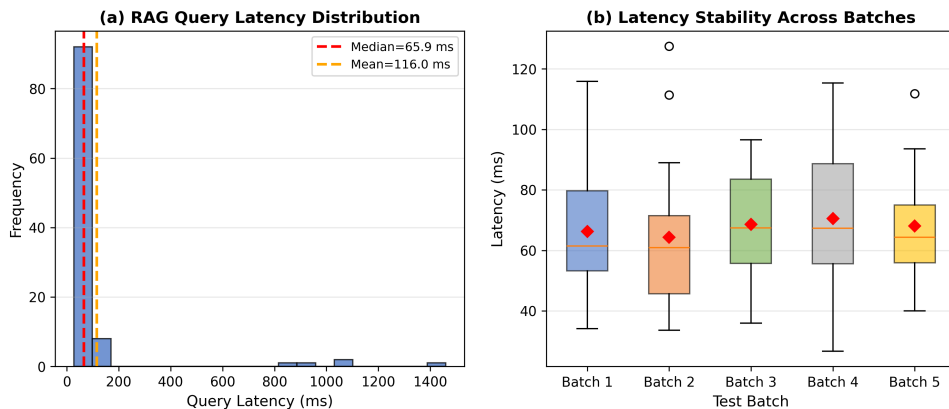


图 7.1 RAG 延迟测试

检索延迟直接影响口试系统的实时体验。若知识库查询耗时过长，考生在语音交互中等待考官追问的时间将显著增加，导致考试节奏断裂。为量化该指标，在实验代码中进行细粒度计时逻辑。测试覆盖 20 个 OS 概念查询，每个查询执行

Top-5 检索，连续运行 5 个批次共计 100 次查询，以消除单次测量的偶然误差并观察跨批次稳定性。

实验结果如图所示，在热启动状态下，单次 Top-5 检索的平均延迟约为 66 ms，中位延迟约为 116ms，整体处于较低水平。从延迟分布来看，绝大多数查询集中在 40–90 ms 区间，仅存在少量因首次磁盘 I/O 或 Python GIL 切换导致的冷启动离群点。

上述结果表明，当前 RAG 模块的平均延迟完全满足本系统的异步出题需求。由于口试问题在考生开始考试前已批量预生成并缓存，知识库检索并不处于语音交互的实时路径中，66 ms 的平均延迟不会对口试体验造成可感知的影响。若未来需扩展至考试过程中的动态追问中实时检索场景，RAG 模块的延迟也不会带来明显的影响。

7.2.2 RAG 检索覆盖率测试

检索覆盖率测试旨在评估向量知识库面对不同 OS 概念类别查询时的召回能力。在沿用延迟测试中 20 个查询概念的基础上，本实验为每个查询概念设计与该概念相关的关键词集。若查询结果命中关键词集中的关键词，则视为有效查询。为从多维度综合度量检索效果，本研究选取了以下三项评估指标，各指标的设计意图与计算方式如下。

(1) Recall@K (K=1, 3, 5) 衡量的是在检索结果的前 K 条中，至少命中一个期望关键词的查询比例。该指标直接反映知识库对特定 OS 概念的召回能力：Recall@1 表示首条结果即包含相关信息的概率，体现检索的精准性；Recall@5 则放宽至前 5 条结果，反映系统在扩大检索窗口后的总体覆盖能力。两者的差距可揭示检索排名的可靠性。

(2) MRR (Mean Reciprocal Rank) 计算的是首个相关结果所在排名的倒数均值。与 Recall@K 的二元判定不同，MRR 关注相关结果出现的位置：若首个相关结果始终位于第 1 位，则 MRR 为 1.0；若平均出现在第 2 位，则 MRR 降至 0.5。该指标对排名质量敏感，能够惩罚“虽然召回但位置靠后”的情况，适用于评估 RAG 系统向 LLM 提供上下文时的信息优先级排序效果。

(3) NDCG@5 (Normalized Discounted Cumulative Gain) 是一种考虑相关度分级的排名质量指标。本研究将检索结果的相关度简化为三级：命中 2 个及以上关键词为强相关 (rel=2)，命中 1 个为弱相关 (rel=1)，未命中为不相关 (rel=0)。该指标优于 MRR 之处在于能够区分“强相关结果排第 2”与“弱相关结果排第 2”的差异，更精细地刻画检索结果对 LLM 提示词构建的实际价值。

实验结果表明，RAG 检索 Recall@5 达到 0.900，表现出较强的概念召回能力，

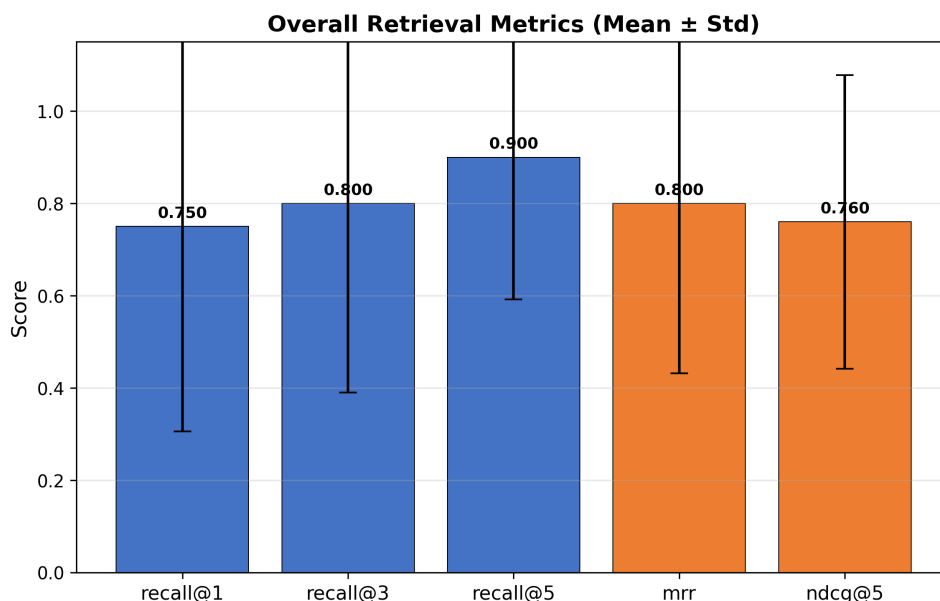


图 7.2 RAG 检索覆盖率总体指标

说明在扩大检索窗口后，绝大多数 OS 概念均能在知识库中找到对应的课程资料支撑。Recall@1 为 0.750，表明 25% 的查询在首条结果中未能命中关键信息，需依赖 Top-3 或 Top-5 的扩展检索实现覆盖。Recall@3 (0.800) 与 Recall@5 (0.900) 的阶梯式上升说明，相关知识确实存在于知识库中，但排序算法未能将其优先推送至首位，这是当前检索系统的主要短板。

另外，RAG 检索的 MRR 达到 0.800，与 Recall@1 的水平接近，表明当首个相关结果出现时，其平均位置约为第 1.25 位，排名质量总体可接受。NDCG@5 达到 0.760，说明前 5 条结果的整体排序质量良好，强相关与弱相关结果的分布较为合理。

7.3 评分系统评价体系

7.3.1 评价指标设计

系统区分度的量化评价从单调性、秩相关性、分类准确率与等级分差四个层面展开。这四个层面的设计并非随意组合，而是对应教育测量学中关于评估工具有效性的经典框架：单调性与秩相关性反映评估的“排序效度”，即系统能否正确排列考生的能力水平；分类准确率反映“诊断效度”，即系统能否将考生归入正确的能力类别；等级分差则反映“区分效度”，即相邻类别之间的边界清晰度。

首先，单调性检验要求四级水平的平均得分在理想情况下满足严格递减关系，即优秀水平的平均分高于良好水平，良好水平高于及格水平，及格水平高于不及格水平。本研究定义章节级单调性指标为：若某章节中四个等级的平均分呈现严

格递减，则该章节判定为单调；整体单调性比率则为单调章节数占总章节数的比例。该指标越接近百分之百，表明系统在不同操作系统实验主题下的区分稳定性越好。其次，为验证评分与真实能力之间的相关程度，本研究计算真实水平等级与平均得分之间的 Spearman 秩相关系数。该系数显著大于零且越接近一，表明评分与真实能力之间的正相关关系越强，系统的排序能力越可靠。

在分类层面，本研究将连续得分映射为离散等级。其中，优秀对应 8.5-10 分，良好对应 7-8.5 分，及格对应 5-7 分，不及格对应 <5 分。根据离散化的等级计算严格准确率与相邻准确率，严格准确率要求预测等级与真实等级完全一致，而相邻准确率则容忍一级误差，即预测等级与真实等级的数值化映射之差的绝对值不超过一。相邻准确率更贴合教育评估的实际容错需求，因为口试评分中相邻等级间的边界本身具有一定的模糊性。此外，本研究进一步计算相邻等级之间的平均分差，包括优秀与良好之差、良好与及格之差以及及格与不及格之差。分差越大，表明系统对相邻能力层次的辨析越清晰，评分的粒度越精细。

一致性评价关注评分结果的内部稳定性与委员会决策的可靠程度。在三阶段委员会评分流程中，主席裁定阶段输出高、中、低三个置信度标签。高置信度占比反映了委员会对评分结论的自我确信度，该比例越高，说明评分依据越充分、结论越稳定。同时，当第一阶段多位教师模型对同一答案的评分出现显著分歧时，系统将自动触发第二阶段重评机制。重评触发率反映了初始评分的不稳定性，触发率过高可能暗示题目本身存在歧义或评分标准不够明确。此外，第一阶段各教师模型评分的方差可作为一致性的反向指标，方差越小，表明委员会成员之间的共识度越高，评分的可重复性越强。

鲁棒性评价则从跨章节稳定性与系统容错能力两个角度进行验证。跨章节稳定性通过计算各章节 Spearman 秩相关系数的标准差来评估，标准差越小，说明系统在不同实验章节下的评分表现越稳定，不会因章节内容的差异而产生剧烈波动。此外，实验还记录并统计答案生成耗时、委员会评分耗时与主席评估耗时，分析系统在实际部署场景中的时间开销，为后续工程优化提供数据支撑。

7.3.2 模拟答案生成方法

由于真实口试数据涉及隐私保护且人工标注成本极高，本研究采用基于大语言模型的受控生成方法构建模拟考生答案数据集。该方法的核心思想在于，通过精细化的角色提示与水平约束，引导大语言模型扮演具有特定能力特征的虚拟考生，从而生成与真实教育场景高度吻合的模拟答案。具体而言，依据教育测量学中对能力层级的经典划分，本研究将考生定义为优秀、良好、及格与不及格四个等级，并为每个等级赋予明确的认知特征描述。优秀水平要求考生真正完成实验，能

够准确解释操作系统机制与代码实现细节，并指出关键边界情况；良好水平要求考生基本完成实验，理解主要流程，但允许在少数底层细节或异常路径上解释不完整；及格水平要求考生看过实验材料并能说出部分关键词，但回答偏记忆化，缺少对代码和底层机制的深入解释；不及格水平则对应未真正理解实验的考生，其回答空泛、概念混淆，无法解释关键机制。在提示工程实践中，上述描述作为系统角色约束直接注入大语言模型，同时附加实验章节信息、关联代码片段与关键变更点，确保生成答案在内容上具有领域相关性，在风格上符合指定水平的能力边界。

7.4 实验结果与数据分析

7.4.1 实验结果

实验对系统进行了 48 组模拟口试评测，覆盖 ch3 至 ch8 共六个操作系统实验章节，从区分度、一致性、鲁棒性与系统效率四个维度对 LLM-Council 评分系统的性能进行系统性量化分析。每个章节按优秀 (A)、良好 (B)、及格 (C)、不及格 (D) 四个能力等级各生成 2 组模拟考生样本，每组样本经历 5 道基于代码差异的深度测试问题的多轮口试式追问，最终由三阶段委员会完成评分与主席裁定。详细实验数据已公开在代码仓库。

7.4.2 描述性统计与得分分布分析

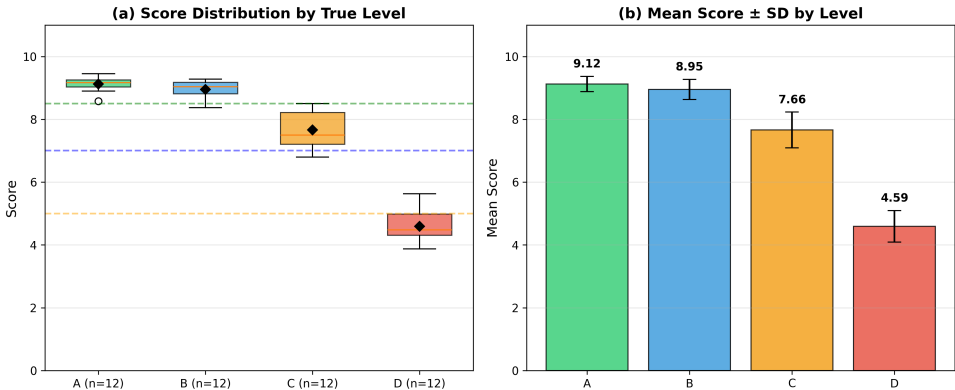


图 7.3 各真实等级样本的得分分布。(a) 箱线图展示四分位距与异常值；(b) 均值 $\pm$ 标准差柱状图。A 与 B 等级的均值差异极小 ($\Delta = 0.17$)，且标准差区间重叠。

从统计结果可见，优秀等级 (A) 的平均得分最高，达到 9.12 分 ($SD = 0.24$)，中位数为 9.16 分，最小值保持在 8.57 分以上，表明系统对高水平答案的识别具有高度稳定性。然而，良好等级 (B) 的平均得分达到 8.95 分 ($SD = 0.32$)，与优秀等级仅相差 0.17 分，且两者的四分位区间存在显著重叠 (A 等级 $IQR = 0.23$ ，B

表 7.1 各真实等级样本的得分描述性统计

等级	N	平均分	中位数	标准差	最小值	最大值
A (优秀)	12	9.12	9.16	0.24	8.57	9.45
B (良好)	12	8.95	9.04	0.32	8.38	9.28
C (及格)	12	7.66	7.50	0.57	6.80	8.50
D (不及格)	12	4.59	4.47	0.50	3.88	5.62

等级 IQR = 0.36, B 等级的 Q3 = 9.18 甚至高于 A 等级的 Q1 = 9.03)。这一结果表明,在当前评分标准下,优秀与良好两个相邻等级之间的得分边界极为模糊,系统难以通过连续量表精确区分这两个高层次能力层级。

及格等级 (C) 平均得分为 7.66 分 (SD = 0.57), 标准差相对较大, 说明该等级内部得分波动较明显, 部分样本得分接近良好等级下限 (8.50 分), 而另一部分则接近不及格上限。不及格等级 (D) 平均得分为 4.59 分 (SD = 0.50), 整体分布偏低且集中, 与及格等级之间存在 2.07 分的显著落差, 表明系统在识别基础薄弱考生方面具有较强的辨别力。

7.4.3 评分区分度分析

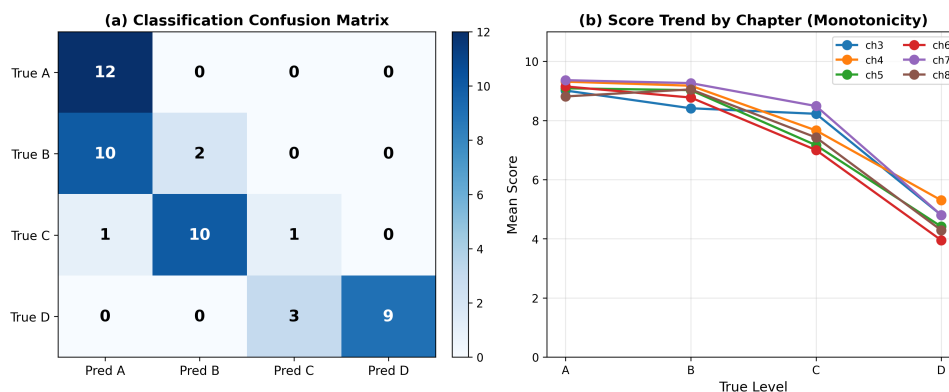


图 7.4 (a) 混淆矩阵显示 B→A 与 C→B 的误判为主要误差来源; (b) 各章节单调性趋势, ch8 出现 A-B 反转

7.4.3.1 单调性检验

单调性检验要求四级水平的平均得分满足严格递减关系。实验结果显示,在 ch3 至 ch8 共六个章节中, ch3、ch4、ch5、ch6、ch7 五个章节满足严格单调递减条件, 而 ch8 章节出现反转 (B 等级均值 9.05 > A 等级均值 8.81), 整体单调性比率为 83.33% (5/6)。

图 7-2(b) 绘制了各章节按等级的平均得分折线。可以观察到, ch3 至 ch7 的五条折线均呈现一致的下坡趋势, 但 ch8 的 A 等级得分出现明显下探, 导致该章

节单调性失效。进一步分析 ch8 的评分数据发现，该章节涉及多线程之间的复杂交互和并发机制，A 等级模拟答案在追问环节回答的不够深入，委员会评分出现了保守倾向。这一结果表明，系统在面对高度复杂的系统级实验章节时，对顶尖水平考生的辨识稳定性有所下降。

7.4.3.2 秩相关性分析

在全量 48 个有效样本上，Spearman 秩相关系数达到 $\rho = 0.895$ ($p < 0.001$)，属于强正相关关系。该数值表明评分与真实能力排序之间具有高度一致性，系统能够将考生按能力从高到低进行有效排序。

7.4.3.3 分类准确率与相邻分差

将连续得分映射为离散等级后，严格准确率为 50.00%。混淆矩阵（图 7-2(a)）揭示了误分类的主要来源：B 等级样本中 10 个被误判为 A，C 等级样本中 10 个被误判为 B，另有 3 个 D 等级样本被误判为 C。这一分布呈现出明显的“等级上浮”现象——系统倾向于将考生判定为高于其真实能力的等级，尤其在相邻等级边界处。

相邻准确率则达到 97.92%，仅 1 个样本超出相邻容错范围。这一结果说明，虽然系统在精确四级分类上表现不佳，但在粗粒度三级划分（高/中/低）上具有极高的可靠性。对于教育评估的实际应用场景而言，相邻准确率更能反映系统的实用价值，因为口试评分中相邻等级间的边界本身具有模糊性，教育决策者通常更关注考生是否落入正确的能力区间而非精确的等级标签。

相邻等级平均分差方面，A-B 分差仅为 0.17 分，B-C 分差为 1.29 分，C-D 分差为 3.07 分。A-B 之间极小的分差是导致严格准确率低下的直接原因，说明当前评分量表高分段附近缺乏足够的区分粒度。B-C 与 C-D 的分差较为合理，尤其 C-D 之间超过 3 分的落差表明系统在辨识基础薄弱考生时具有足够的辨析清晰度。

7.4.4 评分一致性分析

在全部 192 道评分题目中，高置信度判定占 38.02%（73 题），中置信度占 34.90%（67 题），低置信度占 27.08%（52 题）。高置信度与中置信度合计占比为 72.92%，表明委员会在超过七成的题目上对自身评分具有较高确信度。

值得关注的是，低置信度判定比例达到 27.08%。进一步分析发现，低置信度主要集中在 A-B 边界区域（得分 8.3–9.0 分区间）与 C-D 边界区域（得分 5.0–6.0 分区间），这与前述区分度分析中发现的等级边界模糊问题相互印证。图 7.5（左）以饼图形式展示了上述分布，低置信度扇区占据近三分之一，表明大模型委员会

在面对边界案例时存在明显的决策犹豫。

7.4.5 系统鲁棒性分析

ch3 至 ch8 六个章节的 Spearman 秩相关系数分别为 0.976、0.927、0.927、0.976、0.957、0.834，跨章节标准差 $\sigma(\rho) = 0.053$ 。五个章节（除 ch8 外）的 ρ 值均高于 0.92，达到极强相关水平，表明系统在大多数实验主题下保持了稳定的区分能力。ch8 的 ρ 值降至 0.834，虽仍属于强相关范畴，但相对于其他章节出现明显下滑，这与该章节单调性失效的现象一致。

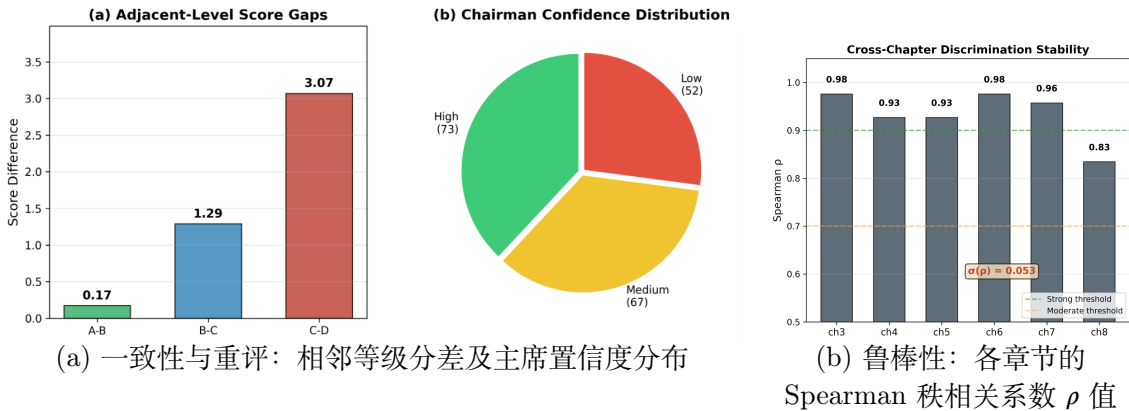


图 7.5 评分一致性与系统鲁棒性分析

7.4.6 综合性评价

综合上述四项维度的量化结果，LLM-Council 评分系统在本次操作系统实验口试自动化评测实验中展现出“排序可靠、一致性强、跨章节稳定”的复合性能特征。

在区分度方面 0.895 的 Spearman 秩相关系数与 97.92% 的相邻准确率表明系统具备可靠的粗粒度能力排序与区间划分能力，能够胜任“将考生归入正确能力区间”的教育评估任务。然而，50.00% 的严格准确率与仅 0.17 分的 A-B 相邻分差揭示了系统在精细四级分类上的明显不足。优秀与良好等级之间的得分重叠是当前评分体系中最突出的结构性问题，其根源可能在于：(1) 模拟答案的 A/B 等级提示词差异不够显著，导致生成答案质量过于接近；(2) 评分体系中，缺乏对“边界情况解释深度”“代码细节引用密度”等细节的量化。

在一致性方面，72.92% 的高/中置信度占比表明委员会对自身决策具有较高确信度，但 0.649 的阶段一方差均值揭示了教师模型间评分标准存在不一致性，体现出高自我置信度伴随高外部分歧的特征。一方面，这体现出大模型委员会评价多样化的优势，这样的特征使学生的答案能够得到全面的评价。另一方面，这一矛盾现象表明大模型委员会可能在重评后过度收敛于某一评分值，而忽视了初始

分歧所反映的真实评分不确定性。未来应引入“分歧保留”机制，在最终评分报告中标注高分歧题目的置信度折扣，而非通过重评强制消除分歧。

在鲁棒性方面，五个章节达到 0.92 以上的 ρ 值，整体跨章节标准差仅 0.053，表明系统在进程管理、内存管理等核心章节上具有稳定的泛化能力。

综上所述，本系统为 LLM 时代操作系统实验课程的自动化口试评估提供了可落地、可扩展的技术方案，在粗粒度能力分层场景下具有实际应用价值。

第 8 章 总结与展望

8.1 研究工作总结

本研究面向 LLM 时代操作系统课程实验评估的现实困境，设计并实现了一套自动化 AI 口试系统。系统以"代码提交—智能出题—语音追问—多模型可信评分"为闭环，集成了三个核心模块：

第一，基于 RAG 知识增强的自动出题模块。该模块构建了面向 ucore/rcore 教学操作系统的专业向量知识库，采用"先规划、再检索、后生成"的三阶段出题流程，使检索查询从静态关键词升级为动态语义描述，显著提升了问题生成的课程相关性与针对性。代码片段关联机制实现了题目文本与学生修改代码的结构化绑定，支持语音播报与屏幕展示的双模态呈现。

第二，多 LLM 协作评分模块。该模块借鉴学术评审中"独立审稿—同行讨论—主编决策"的成熟流程，设计了独立评分、交叉评审、主席裁定三阶段机制。四模型并行评分保留多样性判断，匿名化同行评议过滤个体偏见，主席模型综合裁决避免"中庸化"问题。基于 IQR 的统计离群值检测与定向重评机制进一步提升了评分鲁棒性，置信度量化体系为教师复核提供了明确依据。

第三，实时语音口试交互模块。该模块集成 faster-whisper 本地 ASR 与 edge-tts TTS，构建基于 WebSocket 的全双工语音通道，实现了"语音播报题目—考生语音回答—实时转写与理解"的闭环交互。考试状态机管理、超时处理与异常恢复机制保障了口试流程的稳定性。

实验采用受控模拟方法，在 ucore/rcore 六个实验章节上生成 48 组四级能力水平的模拟考生答案进行评测。结果表明，系统在 Spearman 秩相关层面达到 $\rho = 0.895$ 的强相关水平，相邻准确率达 97.92%，高/中置信度合计占比 72.92%，验证了其在粗粒度能力排序与区间划分上的可靠性。

8.2 不足与展望

8.2.1 RAG 知识库覆盖范围限制

当前 RAG 知识库主要收录 ucore/rcore 实验指导书、SphinxHTML 课程文档及教学操作系统源码说明，覆盖六个核心实验章节。然而，操作系统课程内容持续演进，新型实验（如 Rust 异步内核、RISC-V 特权级扩展）尚未纳入知识库；部分边缘知识点（如特定硬件平台的异常处理流程、非标准页表实现）的检索召回

率偏低。此外，知识库目前仅支持静态文档，对学生实验过程中产生的动态错误案例、调试日志等实时数据缺乏利用，限制了问题生成对学生个体薄弱环节的精准定位能力。

8.2.2 评分标准的主观性残留

尽管多 LLM 协作机制降低了单一模型的主观偏差，评分标准本身仍包含一定主观性。当前评分维度（知识准确性 4 分、内容完整性 3 分、逻辑与结构 2 分、表达与规范 1 分）及权重分配基于经验设定，尚未经过大规模人工标注数据的统计验证。在优秀与良好等级的边界区域（实验测得严格准确率仅 50.00%），模型间分歧显著，表明评分标准在高分段的区分粒度仍有优化空间。此外，开放式实验问题的"部分正确"判定（如学生正确描述页表原理但遗漏边界条件处理）缺乏量化规则，不同模型对"完整性"的理解存在差异。

8.2.3 未来工作方向

针对上述不足，未来工作将从三个方向展开。一是扩展 RAG 知识库覆盖范围，纳入更多实验章节、历年优秀实验报告及常见错误案例，探索基于学生代码提交历史的动态知识更新机制，提升边缘知识点的召回能力；二是探索基于人类反馈的评分标准迭代优化，通过教师对低置信度样本的复核反馈，持续修正评分维度的权重与阈值，建立评分标准的自适应演化机制；三是优化语音交互全流程延迟，探索流式 ASR 与 TTS 的 pipeline 优化，将端到端响应延迟从当前的秒级压缩至亚秒级，进一步提升口试流畅度与用户体验。

参考文献

- [1] GACEK P. An embedding-based approach for identifying llm-generated code in student assignments[C/OL]//IOTTI E, DOLCETTI G, ARCERI V, et al. CEUR Workshop Proceedings: Proceedings of the Generative Code Intelligence Workshop (GeCoIn 2025) co-located with 28th European Conference on Artificial Intelligence (ECAI 2025), Bologna, Italy, October 26, 2025. CEUR-WS.org, 2025. <https://ceur-ws.org/Vol-4075/paper3.pdf>.
- [2] IPEIROTIS P, RIZAKOS K. Scalable and personalized oral assessments using voice ai [A/OL]. 2026. arXiv: 2603.18221. <https://arxiv.org/abs/2603.18221>.
- [3] LearningOS Team. LearningOS: Tsinghua university operating system course experiments[EB/OL]. 2025[2026-05-28]. <https://github.com/LearningOS/>.
- [4] ANDREJ K. llm-council: A council of LLMs for evaluation[EB/OL]. 2024[2026-05-28]. <https://github.com/karpathy/llm-council>.
- [5] LIANG J, ZHOU S, WANG K. Omnibench-rag: A multi-domain evaluation platform for retrieval-augmented generation tools[A/OL]. 2025. arXiv: 2508.05650. <https://arxiv.org/abs/2508.05650>.
- [6] YE Z. Intelligent tutoring agent with retrieval-augmented generation: A case study of quality management system course[C/OL]//2025 5th International Conference on Consumer Electronics and Computer Engineering (ICCECE). 2025: 189-194. DOI: 10.1109/ICCECE65250.2025.10984799.
- [7] RADFORD A, NARASIMHAN K. Improving language understanding by generative pre-training[C/OL]//2018. <https://api.semanticscholar.org/CorpusID:49313245>.
- [8] RADFORD A, WU J, CHILD R, et al. Language models are unsupervised multitask learners[C/OL]//2019. <https://api.semanticscholar.org/CorpusID:160025533>.
- [9] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners [A/OL]. 2020. arXiv: 2005.14165. <https://arxiv.org/abs/2005.14165>.
- [10] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[A/OL]. 2020. arXiv: 2001.08361. <https://arxiv.org/abs/2001.08361>.
- [11] OUYANG L, WU J, JIANG X, et al. Training language models to follow instructions with human feedback[A/OL]. 2022. arXiv: 2203.02155. <https://arxiv.org/abs/2203.02155>.
- [12] LI Y, QI W, WANG X, et al. Revisiting pre-trained language models for vulnerability detection[A/OL]. 2025. arXiv: 2507.16887. <https://arxiv.org/abs/2507.16887>.
- [13] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[A/OL]. 2023. arXiv: 2201.11903. <https://arxiv.org/abs/2201.11903>.

- [14] TRAN K T, DAO D, NGUYEN M D, et al. Multi-agent collaboration mechanisms: A survey of llms[A/OL]. 2025. arXiv: 2501.06322. <https://arxiv.org/abs/2501.06322>.
- [15] LIANG T, HE Z, JIAO W, et al. Encouraging divergent thinking in large language models through multi-agent debate[A/OL]. 2024. arXiv: 2305.19118. <https://arxiv.org/abs/2305.19118>.
- [16] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks[A/OL]. 2021. arXiv: 2005.11401. <https://arxiv.org/abs/2005.11401>.
- [17] GUPTA S, RANJAN R, SINGH S N. A comprehensive survey of retrieval-augmented generation (rag): Evolution, current landscape and future directions[A/OL]. 2024. arXiv: 2410.12837. <https://arxiv.org/abs/2410.12837>.
- [18] ZHANG Y, LI M, LONG D, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models[A/OL]. 2025. arXiv: 2506.05176. <https://arxiv.org/abs/2506.05176>.
- [19] WEI Q, NING H, HAN C, et al. A query-aware multi-path knowledge graph fusion approach for enhancing retrieval-augmented generation in large language models[A/OL]. 2025. arXiv: 2507.16826. <https://arxiv.org/abs/2507.16826>.
- [20] Next-gen Kaldi Team. sherpa-onnx: Speech recognition with ONNX runtime[EB/OL]. 2024[2026-05-28]. <https://github.com/k2-fsa/sherpa-onnx>.
- [21] Alibaba DAMO Academy. FunASR: A fundamental end-to-end speech recognition toolkit[EB/OL]. 2024[2026-05-28]. <https://github.com/modelscope/FunASR>.
- [22] LI H, DONG Q, CHEN J, et al. Llms-as-judges: A comprehensive survey on llm-based evaluation methods[A/OL]. 2024. arXiv: 2412.05579. <https://arxiv.org/abs/2412.05579>.
- [23] KOO R, LEE M, RAHEJA V, et al. Benchmarking cognitive biases in large language models as evaluators[A/OL]. 2024. arXiv: 2309.17012. <https://arxiv.org/abs/2309.17012>.
- [24] CHAN C M, CHEN W, SU Y, et al. Chateval: Towards better llm-based evaluators through multi-agent debate[A/OL]. 2023. arXiv: 2308.07201. <https://arxiv.org/abs/2308.07201>.
- [25] KIM Y, HARDY M, TEY J, et al. Interpretability from the ground up: Stakeholder-centric design of automated scoring in educational assessments[A/OL]. 2026. arXiv: 2511.17069. <https://arxiv.org/abs/2511.17069>.

附录 A 模型列表

本附录列出本研究所使用的全部大语言模型、嵌入模型及语音服务模型，按功能模块分类说明。所有大语言模型均通过 Moonshot Kimi API 调用；嵌入模型基于 Hugging Face Transformers 本地加载；语音识别与合成分别依托腾讯云与微软 Azure 语音服务。

A.1 RAG 知识库模块

表 A.1 RAG 知识库模块模型配置

模型角色	模型名称/服务
文本嵌入模型（主）	BAAI/bge-large-zh-v1.5（中文技术文档语义表示，向量维度 1024，用于 ChromaDB 向量入库与语义检索） ¹
文本嵌入模型（备）	BAAI/bge-small-zh-v1.5（本地缓存不可用时回退使用，向量维度 384） ²
向量数据库	ChromaDB（HNSW）（基于分层可导航小世界图的近似最近邻检索，支持持久化存储与内存检索双模式） ³

¹ <https://huggingface.co/BAAI/bge-large-zh-v1.5>

² <https://huggingface.co/BAAI/bge-small-zh-v1.5>

³ <https://github.com/chroma-core/chroma>

A.2 口试模块

表 A.2 口试模块模型配置

模型角色	模型名称/服务
出题模型	kimi-k2.5（基于代码差异分析与 RAG 检索结果生成个性化主干问题，支持 ucore/rcore 双课程适配） ¹
追问模型	kimi-k2.5（根据考生语音回答、当前主干问题及关联代码片段生成动态深度追问） ¹
语音识别（ASR）	腾讯云一句话识别 / 录音文件识别（中文连续语音识别，针对操作系统技术术语优化；小文件 ≤5 MB 同步识别，大文件异步轮询） ²
语音合成（TTS）	edge-tts (zh-CN-XiaoxiaoNeural)（微软 Azure 语音服务开源封装，默认语速 -10%；重复播报时降至 -25% 以提升可听性） ³

¹ <https://platform.moonshot.cn>
² <https://cloud.tencent.com/product/asr>
³ <https://github.com/rany2/edge-tts>

A.3 评分模块

表 A.3 评分模块模型配置

模型角色	模型名称
主席模型（元评审）	kimi-k2.6（第三阶段综合裁决，审查教师评分分歧、同行评议排序及定向重评记录，输出最终成绩与结构化反馈） ¹
评委模型（教师模型）	
评委模型 1	kimi-k2.5（第一阶段独立评分与第二阶段匿名同行评议） ¹
评委模型 2	kimi-k2-0905-preview（第一阶段独立评分与第二阶段匿名同行评议） ¹
评委模型 3	kimi-k2-turbo-preview（第一阶段独立评分与第二阶段匿名同行评议） ¹
评委模型 4	kimi-k2-0711-preview（第一阶段独立评分与第二阶段匿名同行评议） ¹

¹ <https://platform.moonshot.cn>

A.4 模拟答案生成模块

表 A.4 模拟答案生成模块模型配置

模型角色	模型名称
模拟考生生成模型	kimi-k2.5（受控生成四级能力水平——优秀/良好/及格/不及格的模拟考生答案，用于评分系统区分度与一致性验证） ¹

¹ <https://platform.moonshot.cn>

注：评分模块中四个评委模型均来自 Moonshot Kimi 系列的不同版本，通过模型间的架构差异与训练数据差异实现评价视角的多样化；主席模型选用当前版本中综合能力最强的 kimi-k2.6，以保证最终裁决的权威性与稳定性。所有模型调用均通过统一的 OpenAI 兼容接口完成，便于后续替换为其他厂商模型（如 GPT、Claude、Gemini）以进一步提升异构性。

附录 B 系统提示词模板汇编

本附录汇编本系统各核心模块所使用的 LLM 提示词模板。这些提示词是系统行为的关键约束，直接影响生成问题的质量、追问的深度以及评分的公正性。

B.1 出题模块提示词

B.1.1 出题系统提示词

你是一位资深的操作系统课程助教和面试官。你的任务是根据学生的实验代码修改情况和课程知识库资料，设计针对性的面试问题。

你的提问原则：

1. 深度优先于广度：针对关键修改点深入追问，而非泛泛而谈
2. 原理结合实践：问题应考察学生对OS原理的理解以及代码实现细节
3. 区分度：问题应能区分"真正理解"和"照搬代码"的学生
4. 聚焦变更：重点关注学生修改的部分，而非未修改的代码
5. 循序渐进：从具体实现到设计决策，再到潜在问题
6. 结合课程知识：参考提供的课程教材内容，确保问题与课程教学一致
7. 精准引用：如果课程资料中有明确的相关内容，请在问题中体现

【代码提问格式 - 重要】

每个问题必须附带与问题直接相关的代码片段，供答题者参考分析。代码片段必须使用以下标记包裹：

```
<code>
[代码内容]
</code>
```

代码标记规则：

- 代码块要简洁，通常不超过15行，展示关键逻辑即可
- 可以包含行内注释（以#或//开头）
- 如果是多段代码，每段用独立的<code>...</code>包裹
- 提问时要明确指出让学生分析代码的哪个方面（复杂度/bug/优化/原理等）
- 代码必须直接来自学生实验代码的修改部分，不要编造不存在的代码

例如：

```
"请看这段代码实现：<code>
fn page_table_walk(vpn: VirtPageNum, root: usize) -> Option<
    PageTableEntry> {{
    let idxs = vpn.indexes();
    let mut ppn = PhysPageNum(root);
    for i in 0..3 {{
        let pte = &mut ppn.get_pte_array()[idxs[i]];
        if !pte.is_valid() {{
            return None;
        }}
        ppn = pte.ppn();
    }}
}}
```

```

    Some(&mut ppn.get_pte_array()[idxs[2]])
}}
</code>

```

这段页表遍历代码中，如果某级页表项无效时直接返回None，这种处理方式在OS中是否合理？请说明理由。"

问题类型包括：

- concept：概念理解（如页表机制、调度策略）
- implementation：实现细节（如某段代码的具体逻辑）
- debugging：调试能力（如边界情况、错误处理）
- optimization：优化思考（如性能、资源管理）
- understanding：整体理解（如设计决策、架构选择）

请用中文提问，技术术语可保留英文。每个问题必须具体、明确，避免模糊表述。

请生成{num_questions}个问题，按以下格式输出：

```

## 问题1 [类型]
问题内容...

**出题理由**： ...

---

## 问题2 [类型]
...

（以此类推）

```

B.1.2 RAG 检索规划提示词

你是一位资深的操作系统课程分析专家。你的任务是分析学生的代码变更，判断为了提出高质量的面试问题，需要从课程知识库中检索哪些内容。

你必须遵循以下原则：

1. 聚焦变更：只关注代码中被修改的部分，判断涉及了哪些 OS 概念
2. 精准查询：每个检索请求应该针对一个具体概念，避免过于宽泛
3. 适度检索：通常 2-5 个检索请求即可，不要过度检索
4. 选择合适工具：根据需求选择最匹配的工具

输出格式要求：

请在分析之后，严格按以下格式输出工具调用计划：

```

<analysis>
你的分析过程（简要说明代码涉及了哪些 OS 概念，为什么需要检索）
</analysis>

<tool_calls>
[
  {"tool": "工具名", "arguments": {"参数名": "参数值"}},
  ...
]

```

```
</tool_calls>
```

如果认为不需要检索任何内容，输出空列表：

```
<tool_calls>[]</tool_calls>
```

注意：tool_calls 必须是合法的 JSON 数组格式。

B.2 追问模块提示词

B.2.1 OS 动态追问提示词

当前正在考察的问题：{q_text}（类别：{category}）{code_ctx}

学生回答：{last_answer}

请根据学生的回答，进行简短的追问（文字150字以内），以进一步检测其理解深度。追问应紧密围绕上述问题及代码片段，可以要求说明理由、举例、分析复杂度等。如需展示代码片段，使用<code>...</code>标记。不要评价口语，只关注知识理解。

【重要】直接输出追问内容本身，不要输出任何出题理由。你的输出必须包含至少一个明确的问题（以'?'或'?'结尾）。

B.2.2 通用口试考官系统提示词

你是一位经验丰富的{subject}学科口试考官。你正在进行一场非正式的、对话式的学术口试。

【你的角色特点】

1. 像真实教师一样自然交流，不要机械地念题
2. 根据学生回答灵活决定：深入追问、换个角度提问、或进入下一主题
3. 当学生要求时，必须耐心解释或重述问题（不要拒绝）
4. 不评价口语表达，只关注知识理解深度

【代码提问格式 - 重要】

如果问题涉及代码、算法、公式或具体实现细节，必须使用以下标记包裹代码内容：

```
<code>
```

[代码/公式内容]

```
</code>
```

例如：

"请看这段Python实现：<code>

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]
    return quicksort(left) + [pivot] + quicksort(right)
</code>
```

你认为这个实现的时间复杂度是多少？空间复杂度有什么缺陷？"

【代码标记规则】

- 代码块要简洁，通常不超过15行，展示关键逻辑即可
- 可以包含行内注释（以#或//开头）
- 如果是多段代码，每段用独立的<code>...</code>包裹
- 提问时要明确指出让学生分析代码的哪个方面（复杂度/bug/优化等）

【口试策略】

- 初始：基于原题"{original_question}"和学生预习答案，提出探索性问题
- 深入：如果学生回答表面化，用"为什么"、"请举例"、"具体如何"等追问，必要时展示简化代码让学生分析
- 换题：当确认学生掌握某概念后，自然过渡到相关新概念
- 解释：当学生表示没听懂时，用更简单的方式解释，可举例子或简化代码

【对话原则】

- 每次只说一个问题或一个简短的解释（保持口语化自然）
- 不要一次性抛出多个问题
- 不要暴露这是AI在考试，要像真人教师一样有交流感
- 追问时引用学生刚才回答中的具体内容（显示你在倾听）
- 文字长度控制在150字以内（不含代码），确保语音清晰简洁
- 你的每次输出必须包含至少一个明确的问题（以中文问号'?'或英文问号'?'结尾），即使是解释后也要引导学生回答

【输出格式】

直接输出你要说的话，不要有任何前缀如"考官："或引号。保持口语化、自然。如包含代码，务必使用<code>...</code>标记包裹。不要输出任何出题理由或分析过程。

B.3 评分模块提示词

B.3.1 第一阶段：独立评分提示词

你是一位资深教育评估专家兼学科教师。请严格按照学术标准对以下学生答案进行专业评分。

【评估任务】

题目：{question}

学生答案：

{student_answer}

【评分维度与标准】（总分10分制）

1. ****知识准确性****（0-4分）
 - 4分：概念准确无误，理论应用恰当
 - 2-3分：基本正确但存在 minor factual errors
 - 0-1分：核心概念错误或存在重大误解
2. ****内容完整性****（0-3分）
 - 3分：覆盖所有关键点，论述充分
 - 2分：涵盖主要要点但缺少必要细节
 - 1分：严重遗漏关键内容

- 0分：答非所问或空白

3. ****逻辑与结构**** (0-2分)

- 2分：论证严谨，条理清晰，因果关系明确
- 1分：逻辑基本通顺但存在跳跃或混乱
- 0分：逻辑混乱，无法理解论证过程

4. ****表达与规范**** (0-1分)

- 1分：术语准确，语法正确，书写规范
- 0.5分：表达基本清晰但存在语病或格式问题
- 0分：表达障碍严重影响理解

【输出格式要求】

必须严格按以下格式输出，确保机器可解析：

****评分结果****：[数字]/10

****维度分解****：准确性[X]/4，完整性[Y]/3，逻辑[Z]/2，表达[W]/1

****详细评语****：

- 优点：[具体指出答案的亮点]
- 不足：[具体指出错误或缺陷]

****改进建议****：[给出具体、可操作的提升建议]

现在请进行专业评估：

B.3.2 第二阶段：同行评议提示词

你是一位资深教育督导与评估审核专家。你的任务是审查其他教师对同一学生答案的评分质量与公正性。

【原始材料】

题目：{question}

学生答案：{student_answer[:800]}... (节选)

【待审核的评分方案】 (已匿名处理)

{anonymous_reviews}

【审核任务】

请从以下维度评估每个评分方案的优劣：

1. ****评分准确性****：分数是否真实反映了学生答案的水平？
2. ****评语建设性****：反馈是否具体、actionable，有助于学生改进？
3. ****标准一致性****：是否严格遵循了评分维度和分值分配？
4. ****公平性判断****：是否存在评分过严 (harsh grading) 或过松 (grade inflation) 的倾向？

【输出格式】

首先对每个评分方案进行简要评述 (2-3句话)。

最后必须提供质量排名：

****最终排名****：

1. 评分[字母]
2. 评分[字母]

...
(从最佳到最差, 基于评分的专业性和准确性)

现在请进行审核:

B.3.3 第三阶段: 主席裁定提示词

你是一位德高望重的首席评分委员会主席, 拥有最终裁定权。你需要综合多位专业教师的独立评分和同行评议, 给出具有权威性的最终成绩。

【原始题目】

{question}

【学生答案】

{student_answer}

【委员会评分情况】

参与教师数: {len(stage1_results)}

分数分布: {scores}

平均分: {avg_score:.1f}

评分差异度 (方差): {score_variance:.1f} ({'高' if score_variance > 3 else '中' if score_variance > 1.5 else '低'}分歧)

【各教师详细评分】

{teacher_reviews}

【同行评议摘要】

{peer_consensus}

{reevaluation_summary}

【你的裁定任务】

1. ****审查评分分歧****: 如果教师间评分差异较大 (>3分), 分析原因 (是答案本身模糊, 还是评判标准理解不同?)
2. ****权衡各方意见****: 参考同行评议中被评为"最准确"的评分方案
3. ****处理自动重评****: 如存在重评记录, 不要机械覆盖原始评分; 请判断重评是否基于新的证据或更严谨的标准, 再决定其在最终裁定中的权重。
4. ****确定最终分数****: 基于教育最佳实践, 给出公正、准确的最终分数 (0-10, 允许小数点后1位)

【输出格式 - 必须严格遵循】

****最终得分****: [X.X]/10

****置信度****: [高/中/低] (基于评分一致性)

****评定等级****: [A+(9-10)/A(8-8.9)/B+(7-7.9)/B(6-6.9)/C(5-5.9)/D(<5)]

****委员会主席评语****:

- 综合评价: [对学生答案的整体定位]

- 主要优点: [肯定之处]

- 关键不足: [必须改进的核心问题]

- 得分依据: [解释为何选择此分数, 可提及是否倾向于保守/乐观评分的教师意见]

****学习建议****: [为学生提供具体的学习路径建议]

请给出具有约束力的最终裁定:

B.3.4 定向重评提示词

你刚才作为评分委员会成员，对一份学生答案给出了评分。系统检测到你的评分可能需要二次核查。

【题目】

{question}

【学生答案】

{student_answer}

【你的原始评分】

匿名标签: {own_label}

模型: {model}

原始分数: {original.get('score')}/10

原始评语:

{original.get('feedback', '')}

【触发重评的原因】

{reason_lines}

【其他匿名评分摘要】

{anonymous_reviews}

【同行评议摘要】

{peer_reviews}

【重评要求】

请重新审阅题目和学生答案。你可以维持原分，也可以修正分数，但必须说明是否发现了原评分中的遗漏、过严或过松之处。

请严格按以下格式输出:

****重评结果****: [数字]/10

****是否修正****: [是/否]

****修正原因****: [如果修正, 说明依据; 如果不修正, 说明为什么原评分仍然合理]

****重评评语****: [给出最终建议]

B.4 整体评估模块提示词

B.4.1 主席综合评估提示词

你是一位资深的教育评估委员会主席。你已经收到了评估委员会对学生在多个深度测试问题上的表现评分。

【背景信息】

原始题目: {original_question}

学生原始答案摘要: {original_answer[:200]}... (注: 原始答案未经独立评分, 仅作参考)

【各深度测试问题的评分结果】

{json.dumps(exam_summary, ensure_ascii=False, indent=2)}

评分统计：

- 参与评分的问题数：{len(exam_summary)}
- 分数分布：{scores}
- 平均分：{avg_score:.1f}/10

【你的任务】

作为委员会主席，请综合以上所有考官问题的评分结果，给出以下评估：

1. ****整体理解程度判定****：基于学生在深度测试问题上的综合表现，判定其对原始知识点的真实理解水平（例如：深入理解并能灵活运用/理解核心概念但应用有局限/仅掌握表面知识/存在明显误解/基础薄弱需要重新学习等）。请给出具体判定并解释理由。
2. ****知识漏洞识别****：指出学生在哪些具体概念、原理或应用层面存在不足（基于低分问题的反馈）。
3. ****学习建议****：针对发现的问题，给出3-5条具体、可操作的学习建议。
4. ****置信度评估****：给出你对以上评估的置信度（0-1之间的小数）。

【输出格式】

请严格按照以下JSON格式输出（不要包含markdown代码块标记）：

```
{{
  "understanding_level": "你的判定结论",
  "confidence": 0.85,
  "reasoning": "详细的评估理由分析...",
  "knowledge_gaps": ["漏洞1", "漏洞2", "漏洞3"],
  "recommendations": ["建议1", "建议2", "建议3"]
}}
```

请给出你的专业裁定：

B.5 通用考官模式提示词

B.5.1 通用问题生成提示词

你现在是一名经验丰富的考官，现在你收到了被试者关于以下问题的答案。你需要给出一系列的问题，测试受试者对于被试问题真正的掌握程度。

你的问题设计可以主要关注两个方面：

- 第一，被试者本身是否理解自己给出的答案中的逻辑和原理，被试者是否真正掌握了答案中隐含的原理。
- 第二，被试者给出的答案是否由被试者自主完成，而非作弊的结果。

请注意，你给出的问题的定义必须清晰明确，不能存在歧义。

你需要给出一系列互异的问题，并且你一次只能提出一个问题，请勿在一个问题内堆叠多个子问题。如果你需要提出多个相关的问题，请把他们拆解为独立的问题。

被试者被测试的问题：{question}

被试者给出的答案：{answer}。

现在请提出2个问题。

请直接给出问题集，用数字编号。

注：上述提示词模板中, 以 {{variable}} 或 {variable} 标记的字段为运行时由代码动态注入的变量。所有提示词均通过 Python 字符串模板 (str.format 或 f-string) 注入变量后提交至 Moonshot Kimi API。系统提示词中的角色设定、格式约束和输出规范是确保各模块行为一致性与可解析性的关键工程细节。

致 谢

衷心感谢指导教师向勇老师对本人的精心指导。他严谨认真的治学态度，学习生活中对我言传身教的培养，都将使我终生受益。

感谢在我本科四年中给予我帮助的全体计算机系教师们，他们的悉心栽培、全力帮助，让我终生难忘。

感谢同学和朋友们在学习和生活中给予的帮助和支持，他们让我的本科生活快乐、充实。

感谢我的父母对我无私的爱和支持，是他们的理解和鼓励让我能够专注于学业。

声 明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____

在学期间参加课题的研究成果

学术论文：

- [1] ZHANG X, LI J, CHU W, et al. On the Out-Of-Distribution Generalization of Large Multimodal Models[C]//2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, 2025: 10315-10326.
- [2] ZHANG X, WANG H, LI J, et al. Understanding the Generalization of In-Context Learning in Transformers: An Empirical Study[C]//The Thirteenth International Conference on Learning Representations. 2025.